ABSTRACT

The **Student Portal** is a centralized web application designed to streamline academic administration and enhance communication between students, teachers, and administrators. The platform leverages **machine learning algorithms**, specifically **Naïve Bayes** and **XGBOOST**, to predict student performance based on historical academic data. With XGBOOST achieving over **93% accuracy**, the system enables early intervention and informed decision-making. The portal consists of three user modules: **Admin**, **Teacher**, and **Student Dashboards**, each tailored with specific functionalities.

TABLE OF CONTENTS

01	**INTRODUCTION**	**1**
02	**LITERATURE SURVEY**	**2 - 3**
03	**SYSTEM ANALYSIS**	
	Existing system	**4**
	Proposed system	**5**
	Feasibility study	**6 - 7**
04	**SYSTEM SPECIFICATIONS**	
	Functional Requirements Non-functional Requirements	**8 - 9**
	Hardware Requirements Software Requirements	**10**
05	**SYSTEM DESIGN**	**11**
	System Architecture	**12**
	Data Flow Diagram	**13**
	UML Diagrams	**14 - 17**
06	**SYSTEM IMPLEMENTATION**	**18**
	Project Modules	**19**
	Methodology (Algorithms)	**21**

TABLE OF CONTENTS

	Tools & Technologies	22 - 23
	Sysstem Implementaion	24- 30
	Source code	31 - 45
07	**SYSTEM TESTING**	46
	Testing Methods	47 - 48
	Test Cases	49 - 50
08	**EXPERIMENTAL RESULTS**	51 - 65
09	**FUTURE SCOPE**	66
10	**CONCLUSION**	67
11	**BIBLIOGRAPHY**	68

INTRODUCTION

The Student Portal is a centralized web-based application developed to simplify academic management and improve student performance monitoring. The system integrates Machine Learning (ML) techniques to predict student performance using historical academic data. Two Machine Learning algorithms, Naïve Bayes and XGBoost, are used to analyse student marks and generate performance feedback such as “Good,” “Fair,” and “Poor.” Among these algorithms, XGBoost achieved higher prediction accuracy of more than 93%.

The student portal provides separate dashboards for Admins, Teachers, and Students, allowing them to manage academic activities efficiently. Teachers can manage attendance, upload study materials, create assignments, enter marks, and communicate with students and parents. Students can access study materials, view assignments, check marks, and receive performance feedback by the ML models.

The system improves communication, reduces manual work, and provides data driven insights into student academic performance. By integrating Machine Learning with academic management, the Student Portal helps educational institutions improve efficiency, monitoring, and overall educational outcomes.

LITERATURE SURVEY

Student Portal Systems and Educational Data Mining has been conducted to understand current technologies, methodologies, and challenges in academic management software. Various studies were analysed to gather ideas and improve the efficiency of student performance monitoring systems using Machine Learning techniques.

1. Student Performance Prediction using Machine Learning

Authors: Romero, C., & Ventura, S.

Description:
This study focuses on the use of Machine Learning techniques for analysing student academic data and predicting performance outcomes. The research highlights the importance of Educational Data Mining in improving teaching methods and identifying weak students at an early stage.

2. Educational Data Mining and Learning Analytics

Authors: Baker, R. S. J. d., & Inventado, P. S.

Description:
This paper explains how learning analytics and data mining techniques can improve academic decision-making. The study discusses predictive models that help educational institutions monitor student progress and improve learning outcomes.

3. Predicting Student Performance using Machine Learning Techniques

Authors: Kotsiantis, S. B., Pierrakeas, C. J., & Pintelas, P. E.

Description:
This research presents the use of Machine Learning algorithms for predicting student performance in educational systems. The authors compare different predictive models and emphasize the effectiveness of intelligent academic monitoring systems.

4. XGBoost: A Scalable Tree Boosting System

Authors: Chen, T., & Guestrin, C.

Description:
This paper introduces the XGBoost algorithm and explains its efficiency in classification and prediction tasks. The study highlights its high accuracy and scalability, making it suitable for student performance prediction systems.

5. Data Mining Applications for Student Performance Analysis

Authors: Yadav, S. K., Bharadwaj, B., & Pal, S.

Description:
This study compares different data mining approaches used for student performance analysis. It discusses the role of predictive analytics in improving academic performance and supporting educational management systems.

Conclusion

The reviewed literature highlights the importance of web-based Student Portal Systems and Machine Learning techniques in improving academic management and student performance analysis. Most studies emphasize the use of predictive models, centralized academic systems, and educational data mining for efficient monitoring and decision-making. These insights helped in developing the proposed Student Portal System using Naïve Bayes and XGBoost algorithms for accurate student performance prediction and academic management.

SYSTEM ANALYSIS

The Student Portal System using Machine Learning is a web-based application developed to improve academic management and student performance monitoring. The system provides a centralized platform for Admins, Teachers, and Students to manage attendance, assignments, marks, study materials, and communication efficiently.

EXISTING SYSTEM: -

In traditional student management systems, academic activities are mostly managed manually using spreadsheets, paper records, and emails. Teachers and administrators handle attendance, assignments, and marks separately, which increases workload and chances of errors. These systems do not provide intelligent analysis or prediction of student performance.

Communication between students, teachers, parents, and administrators is also limited and not well integrated. As a result, institutions face difficulties in monitoring student progress effectively and providing timely academic support.

Disadvantages of Existing System: -

1. Traditional systems provide only marks and records without any predictive analysis or intelligent feedback about student performance.
2. Existing systems are not centralized, requiring different platforms or manual methods for managing students, teachers, attendance, and assignments.
3. Manual record management increases paperwork, consumes more time, and may lead to data errors.
4. Communication between stakeholders is limited and lacks real-time interaction.

PROPOSED SYSTEM: -

The proposed Student Portal System is a centralized web-based application developed using Machine Learning techniques for efficient academic management and student performance prediction. The system includes separate dashboards for Admins, Teachers, and Students to manage attendance, assignments, marks, study materials, and communication.

Machine Learning algorithms such as Naïve Bayes and XGBoost are used to analyse historical student marks and predict performance outcomes like "Good," "Fair," and "Poor." The portal improves transparency, reduces manual work, and helps teachers identify weak students at an early stage. XGBoost achieved more than 93% prediction accuracy, making the system effective for data-driven educational management.

Advantages of Proposed System

1. The system provides a centralized platform for managing academic activities of students, teachers, and administrators.
2. Machine Learning algorithms help predict student performance accurately and provide intelligent feedback.
3. The portal reduces manual work and improves efficiency in attendance, assignment, and marks management.
4. Integrated communication features improve interaction between students, teachers, and parents.
5. The system supports better academic monitoring and decision-making through data-driven insights.

FEASIBILITY STUDY

The feasibility study of the proposed Student Portal System using Machine Learning evaluates the practicality and viability of developing and implementing the system using technologies such as Python, Django, MySQL, and Machine Learning algorithms. The analysis confirms that the proposed system is technically, economically, and operationally feasible.

The system is designed to improve academic management and student performance monitoring by integrating Machine Learning models such as Naïve Bayes and XGBoost for predictive analysis and feedback generation.

The feasibility analysis ensures that the proposed system can be successfully developed and implemented within the available resources, technology, and organizational environment.

1. Technical Feasibility: -

Technical feasibility evaluates whether the required technology, hardware, software, and technical resources are available to develop the proposed system. The Student Portal System is technically feasible because it is developed using technologies such as Python, Django, HTML, CSS, and MySQL. Machine Learning libraries like Scikit-learn and XGBoost are open-source and easy to integrate.

2. Economic Feasibility: -

Feasibility determines whether the proposed system is cost-effective compared to the existing system. The Student Portal System is economically feasible because most development technologies are open-source and free. The system reduces paperwork, minimizes manual work, and saves time for administrators and teachers.

3. Operational Feasibility: -

Operational feasibility evaluates whether the system can function effectively within the institution. The proposed Student Portal System is operationally feasible because it provides user-friendly dashboards for Admins, Teachers, and Students. The system simplifies attendance management, assignment handling, communication, and performance monitoring.

4. Schedule Feasibility: -

Schedule feasibility determines whether the project can be completed within the planned time. The proposed project is schedule feasible because the required technologies and development tools are readily available. The project follows a structured development process, and the modules are clearly organized, reducing development time.

Conclusion: -

Feasibility analysis confirms that the proposed Student Portal System using Machine Learning is technically, economically, operationally, and schedule-wise feasible The system provides an efficient and intelligent solution for academic management and student performance prediction using Naïve Bayes and XGBoost algorithms.

SYSTEM SPECIFICATIONS

The proposed Student Portal System is a web-based application developed using Python, Django, and MySQL. The system operates on a client-server architecture and can be accessed through standard web browsers. The frontend is developed using HTML, CSS, and JavaScript to provide a simple and user-friendly interface, while the backend handles academic management, authentication, and Machine Learning operations. The system supports role-based access for Admins, Teachers, and Students.

Functional Requirements

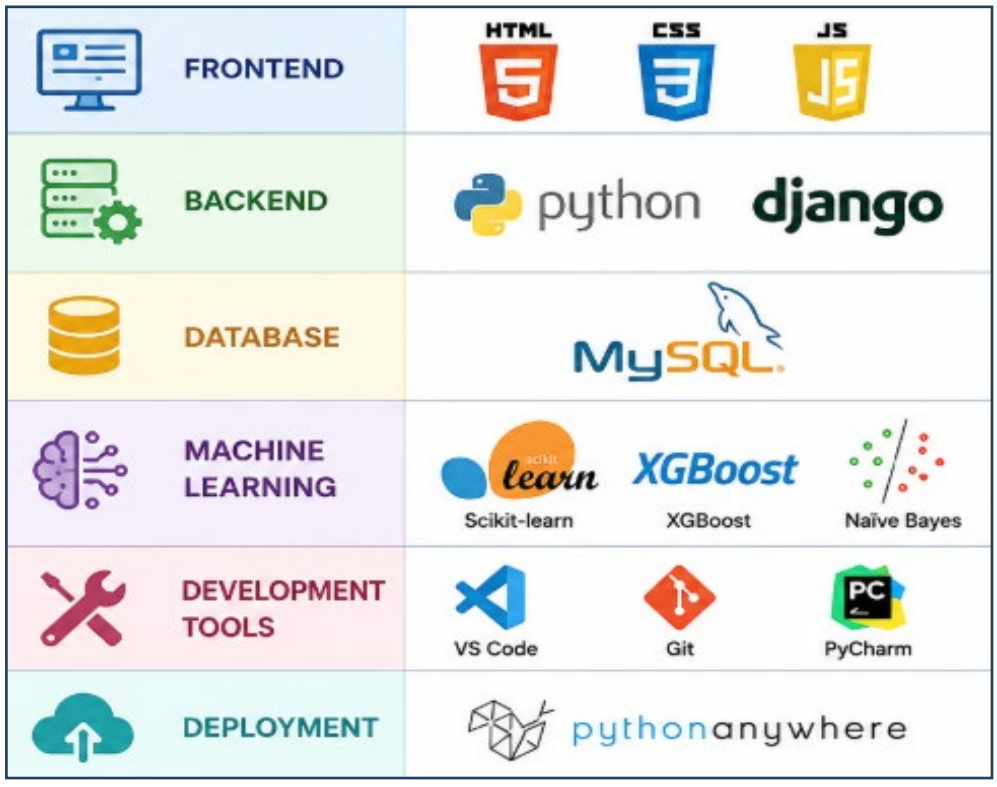
The Student Portal System should allow Admins, Teachers, and Students to securely log in using valid credentials. The admin should be able to add and manage teacher and student details. Teachers should be able to mark attendance, upload study materials, create assignments, add marks, and communicate with students and parents. Students should be able to view assignments, download materials, send messages, and check marks with Machine Learning based performance feedback. The system should also predict student performance using Naïve Bayes and XGBoost algorithms.

Non-Functional Requirements

The system should provide a user-friendly interface for all users. It must ensure data security and maintain user privacy through authentication mechanisms. The application should provide fast response time and support multiple users simultaneously. The system should be reliable, scalable, and easy to maintain. It should also support future enhancements and Machine Learning model updates.

Technology Stack Overview

The Technology Stack Overview provides information about the technologies used in developing the Student Portal System. The system uses Python and Django for backend development, HTML, CSS, and JavaScript for frontend design, and MySQL for database management. Machine Learning libraries such as Scikit-learn and XGBoost are used for student performance prediction and analysis. These technologies together provide a secure, scalable, user-friendly, and high-performance academic management system.



Software Component	Requirement
Operating System	Windows 10/11 or Linux
Frontend	HTML, CSS, JavaScript
Backend	Python 3.13.12, Django
Database	MySQL
ML Libraries	Scikit-learn, XGBoost
Browser	Google Chrome / Microsoft Edge
IDE	VS Code or PyCharm

Hardware Component	Requirement
Processor	Intel Core i3 or above
RAM	4 GB minimum
Hard Disk	500 GB
Monitor	15” LED Monitor
Input Devices	Keyboard and Mouse
Network	Internet Connection

SYSTEM DESIGN

The system design defines the overall architecture, workflow, and structural representation of the Student Portal System using Machine Learning. It provides a blueprint for developing the system by illustrating how different modules interact and how academic data is processed within the application.

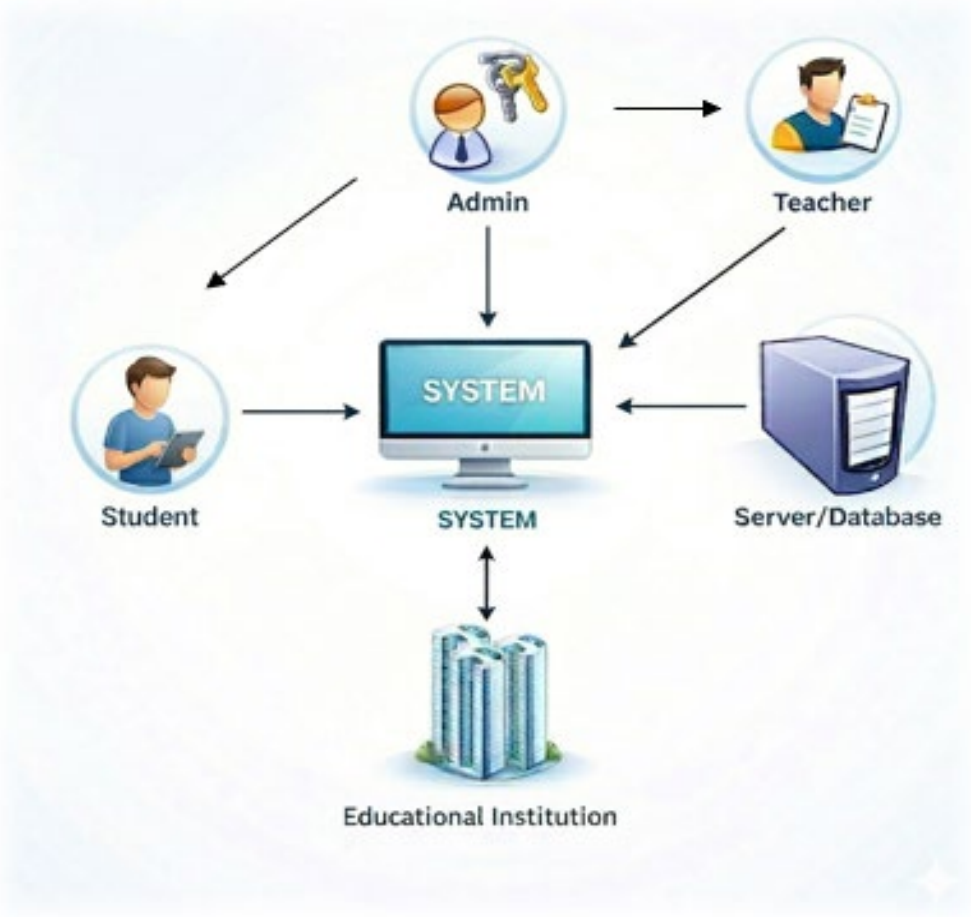
The system consists of separate modules for Admins, Teachers, and Students. The admin module manages teacher and student records, the Teacher module handles attendance, assignments, marks, study materials, while

Student module allows users to access academic information and communicate with faculty. The system also integrates Machine Learning algorithms such as Naïve Bayes and XGBoost to analyse historical student data and predict academic performance.

The design ensures smooth data flow between modules, secure data management, user authentication, and efficient communication among stakeholders. The centralized architecture improves scalability, reliability, and overall academic management efficiency.

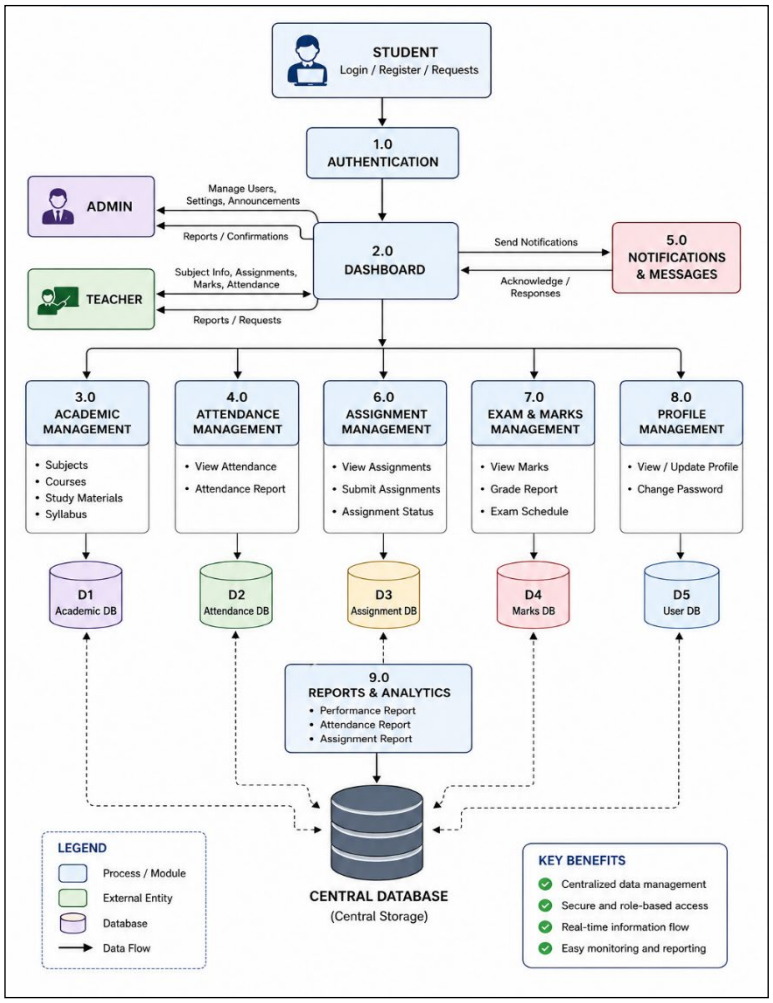
System Architecture

The System Architecture of the Student Portal System shows the interaction between Admin, Teacher, Student, Server/Database, and the Educational Institution through a centralized system. Admins manage users and academic activities, Teachers update attendance, assignments, and marks, while Students access study materials and academic information. All data is stored and managed through the server/database ensuring secure and efficient communication between all modules of the system.



Data Flow Diagram

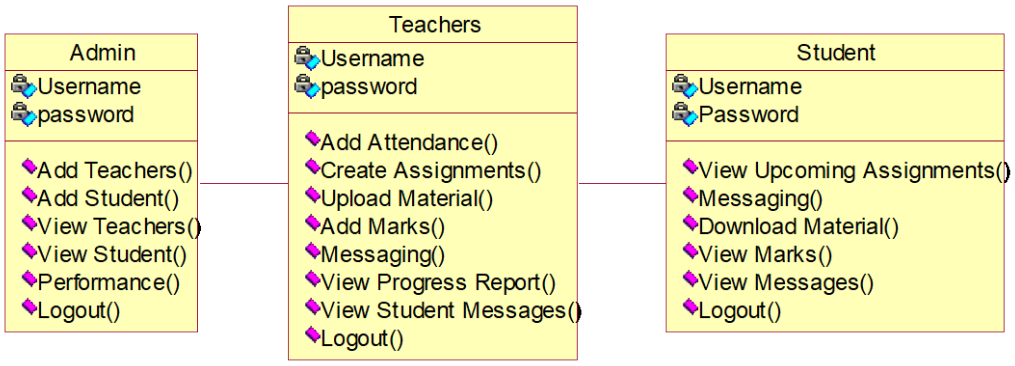
The Data Flow Diagram (DFD) of the Student Portal System shows how data moves between users, modules, and the database. Admins, Teachers, and Students interact modules, Authentication, Academic Management, Attendance, Assignments, and Marks Management. The system stores and manages all information centralized database, ensuring smooth and secure academic operations.



UML Diagrams

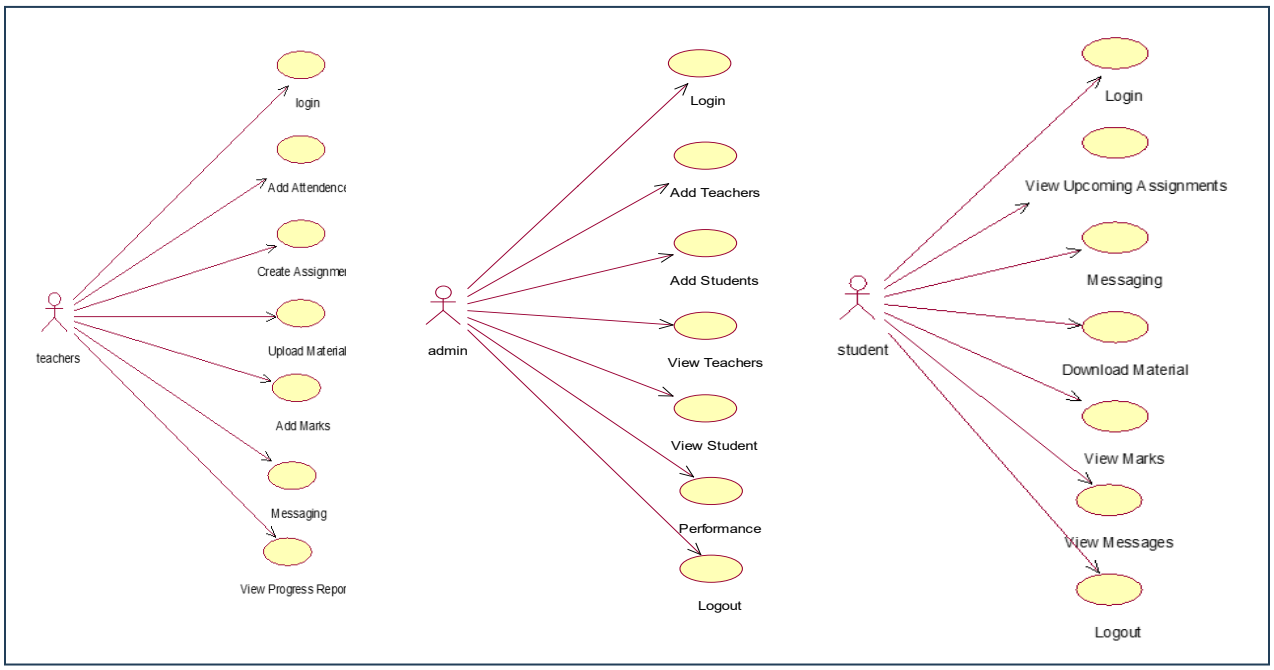
CLASS DIAGRAM: -

The class diagram is used to refine the use case diagram and define a detailed design of the system. The class diagram classifies the actors defined in the use case diagram into a set of interrelated classes. The relationship or association between the classes can be either an "is-a" or "has-a" relationship. Each class in the class diagram may be capable of providing certain functionalities. These functionalities provided by the class are termed "methods" of the class.



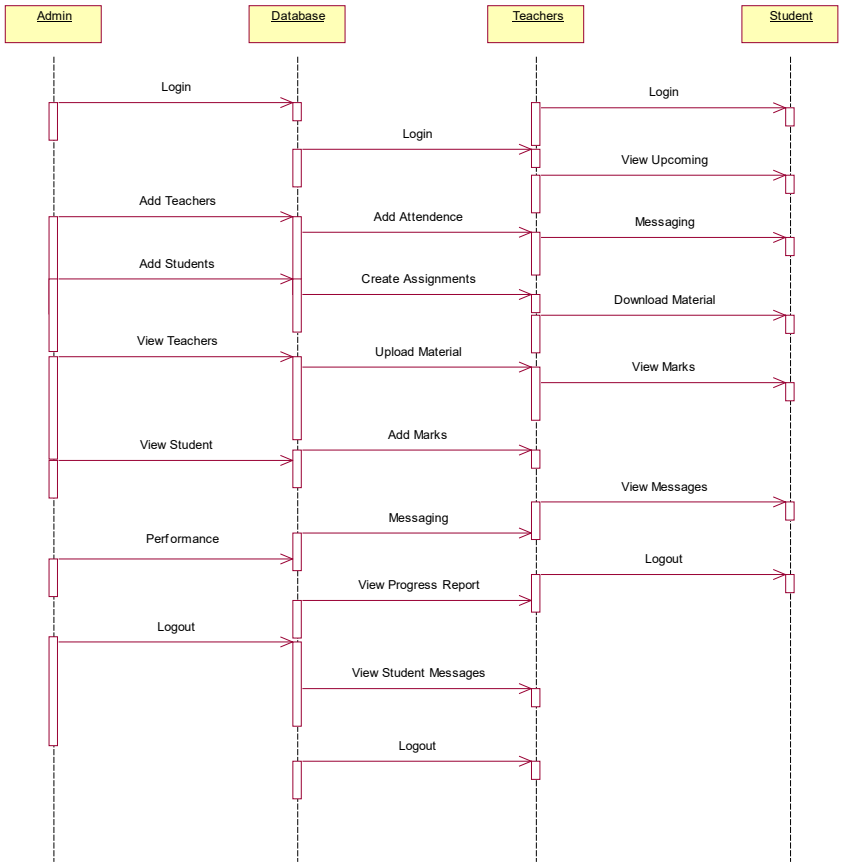
Use case Diagram: -

A use case diagram in the Unified Modelling Language (UML) is a type of behavioural diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.



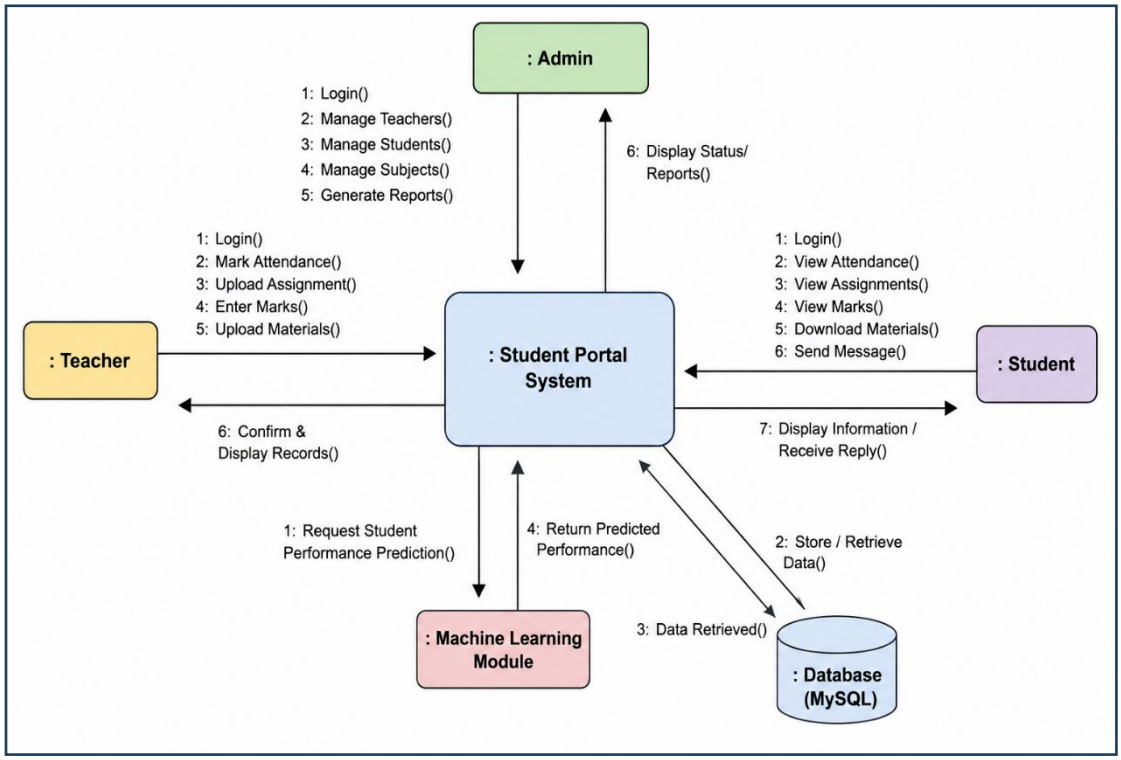
Sequence Diagram:

A sequence diagram represents the interaction between different objects in the system. The important aspect of a sequence diagram is that it is time-ordered. This means that the exact sequence of the interactions between the objects is represented step by step. Different objects in the sequence diagram interact with each other by passing "messages".



Collaborative Diagram:

A collaboration diagram groups together the interactions between different objects. The interactions are listed as numbered interactions that help to trace the sequence of the interactions. The collaboration diagram helps to identify all the possible interactions that each object has with other objects.



System Implementations

The Student Portal System is developed as a web-based application using Python and the Django framework to simplify and automate academic management activities. The system provides separate dashboards for Admins, Teachers, and Students, allowing each user to access features based on their roles and responsibilities.

Admins can manage users, courses, attendance, and academic records, while Teachers can upload assignments, study materials, attendance, and marks. Students can view their academic performance, download materials, submit assignments, and receive notifications through the portal.

MySQL is used as the centralized database for securely storing and managing academic data such as student records, attendance, marks, assignments, and user details. The system ensures secure access through authentication and role-based permissions, providing smooth and reliable data flow across all modules.

In addition, Machine Learning algorithms such as Naïve Bayes and XGBoost are integrated into the system to analyze student academic data and predict performance. These algorithms help identify students who may require additional academic support based on factors like attendance, marks, and assignment performance.

The system improves efficiency, reduces manual work, enhances communication, and provides a user-friendly platform for effective academic management.

Project Modules

The Student Portal System contains separate modules for Admins, Teachers, and Students. The Admin Dashboard manages teacher and student details, subjects, timetables, and reports. The Teacher Dashboard allows faculty members to manage attendance, assignments, study materials, marks, and student communication.

The Student Dashboard enables students to view attendance, assignments, study materials, marks, and Machine Learning based performance feedback. These modules work together to provide centralized and efficient academic management.

Key Features by Role: -

Login Credentials: admin / admin

Core Modules

- Manage Teachers
- Manage Students
- Manage Subjects and Courses
- Manage Timetable
- View Reports and Analytics
- Manage Notifications and Announcements
- Monitor System Activities

Teacher Dashboard

Login: Credentials provided by Admin

Core Modules

- Mark Attendance
- Upload Study Materials
- Create and Manage Assignments
- Enter Student Marks
- View Student Performance
- Send Notifications and Messages
- Generate Academic Reports

Student Dashboard

Login: Credentials provided by Admin

Core Modules

- View Attendance
- View Assignments
- Download Study Materials
- View Marks and Reports
- Receive Performance Feedback
- View Timetable
- Send Messages and Notifications

Methodology (Algorithms)

The Student Portal System uses Machine Learning algorithms to predict student performance using historical academic marks data. The process includes collecting student data, training the Machine Learning models, and generating performance predictions. Two algorithms are used in the system:

1. Naïve Bayes Algorithm

Naïve Bayes is a classification algorithm that predicts student performance based on probability. It classifies students into categories such as “Good,” “Fair,” and “Poor” using academic records.

2. XGBoost Algorithm

XGBoost is an advanced Machine Learning algorithm used for accurate prediction and analysis. It processes student marks efficiently and provides better prediction accuracy. In this project, XGBoost achieved more than 93% accuracy in predicting student performance.

These Machine Learning models are integrated into the Student Portal System to provide real-time feedback and improve academic decision-making.

Tools & Technologies

Python

Python is the main programming language used for developing the Student Portal System. It is simple, easy to learn, and supports object-oriented programming. Python is widely used for web development and Machine Learning applications because of its readability, flexibility, and large collection of libraries.

Django

Django is a Python-based web framework used for backend development. It helps in developing secure, scalable, and fast web applications. Django manages server-side operations, user authentication, database connectivity, and application logic.

HTML, CSS, and JavaScript

HTML, CSS, and JavaScript are used for designing the frontend interface of the Student Portal System. HTML structures the web pages, CSS improves the design and styling, and JavaScript adds interactivity and dynamic functionality.

MySQL

MySQL is used as the database management system for storing student details, attendance records, assignments, marks, and academic information securely.

Scikit-learn

Scikit-learn is a Machine Learning library used for implementing algorithms such as Naïve Bayes and XGBoost. It helps in training models and predicting student performance.

XGBoost

XGBoost is an advanced Machine Learning algorithm used for accurate student performance prediction. It provides high prediction accuracy and efficient data processing.

VS Code / PyCharm

VS Code and PyCharm are used as development environments (IDEs) for coding, debugging, and managing the project efficiently.

Web Browser

Google Chrome and Microsoft Edge are used to access and run the Student Portal System through a web interface.

System Implementation

Install Python Step-by-Step in Windows

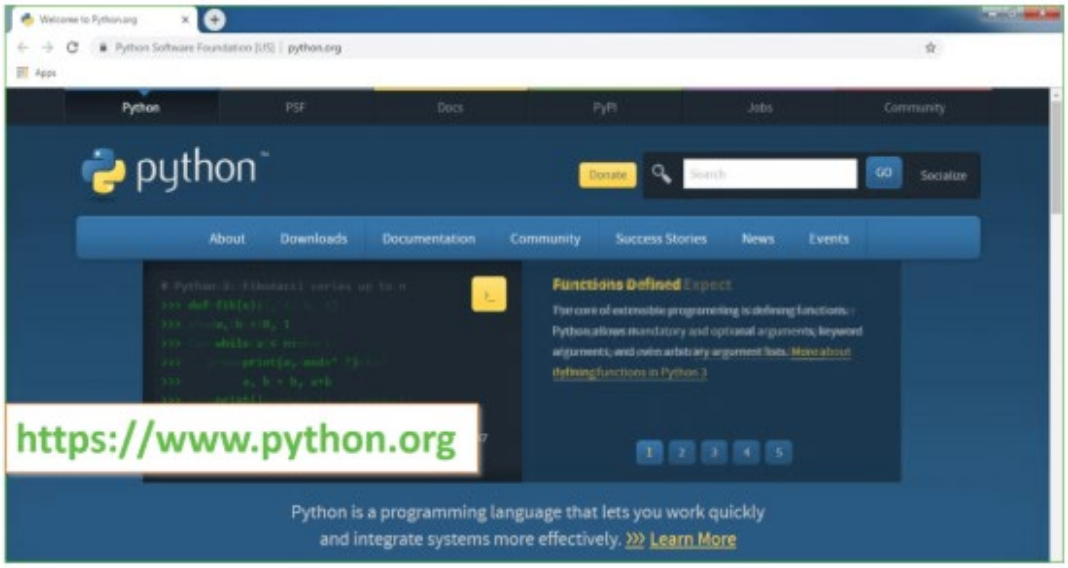
Python is a versatile and widely used high-level programming language that is used for web development, Machine Learning, Artificial Intelligence, and software applications. Python was first released in 1991 and is known for its simple syntax, readability, and object-oriented programming features.

Helps programmers write clear efficient code for projects. Python does not come pre-installed on Windows systems, so it must be downloaded and installed manually.

Over the years, several versions of Python have been released, with Python 3 being the latest and most commonly used version. Before starting the installation process, users should check their system requirements such as operating system type and processor architecture (32-bit or 64-bit).

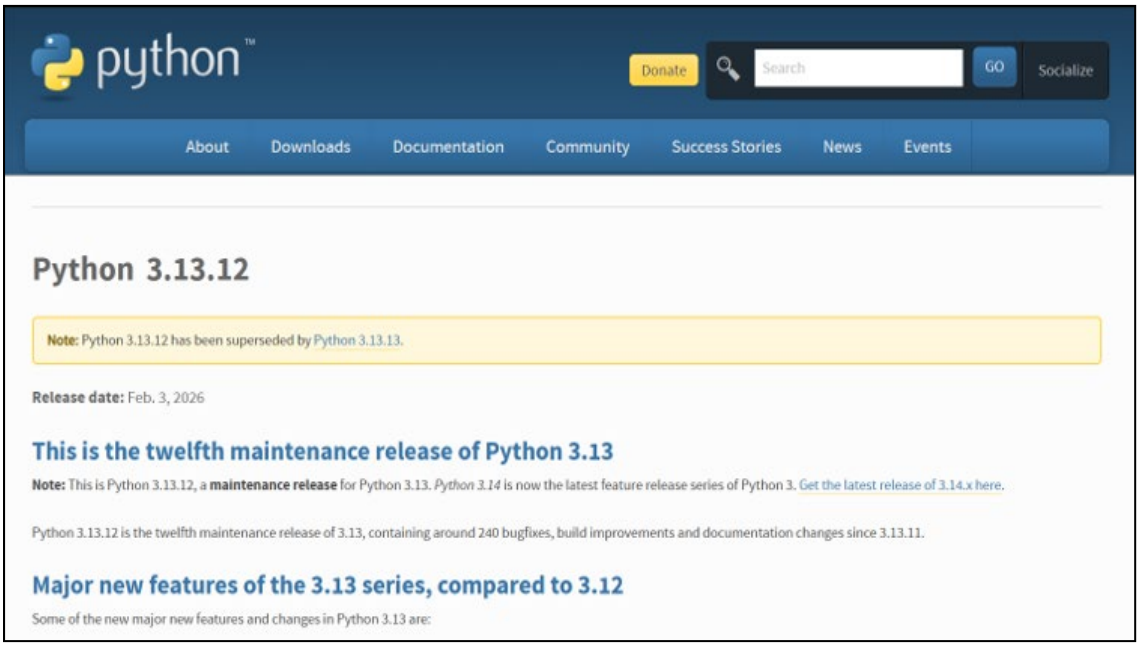
Based on the system configuration, the correct version of Python should be downloaded from the Python website. After users need to run the setup file, select the “Add Python to PATH” option, and complete the installation process. Once installed, Python can be verified successfully using Command Prompt or Terminal.

Step 1: Go to the official site to download and install python using Google Chrome or any other web browser. OR Click on the following link: https://www.python.org

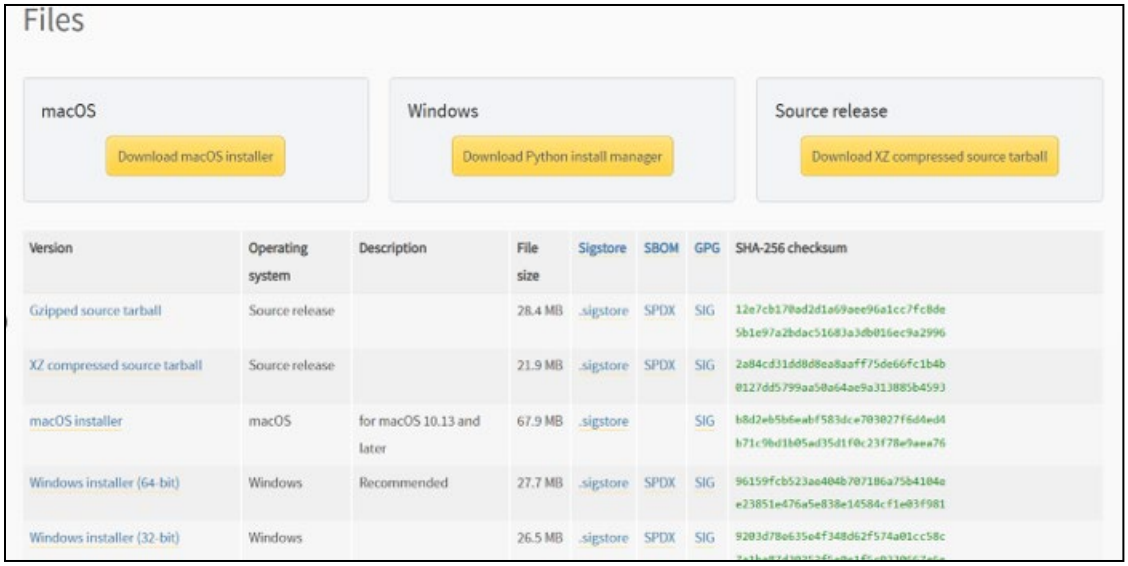


Now, check for the latest and the correct version for your operating system.

Step 2: Click on the Download Tab.



Step 3: You can either select the Download Python for windows 3.13.12 button in Yellow Color or you can scroll further down and click on download with respective to their version.



Files

macOS

Download macOS installer

Windows

Download Python install manager

Source release

Download XZ compressed source tarball

Version	Operating system	Description	File size	Sigstore	SBOM	GPG	SHA-256 checksum
Gzipped source tarball	Source release		28.4 MB	.sigstore	SPDX	SIG	12e7cb170ad2d1a69aee96a1cc7fc8de5b1e97a2bdac51683a3db016ec9a2996
XZ compressed source tarball	Source release		21.9 MB	.sigstore	SPDX	SIG	2a84cd31dd8d8ea8aaff75de66fc1b4b0127dd5799aa50a64ae9a313885b4593
macOS installer	macOS	for macOS 10.13 and later	67.9 MB	.sigstore		SIG	b8d2eb5b6eabf583dce703027f6d4ed4b71c9bd1b05ad35d1f0c23f78e9aea76
Windows installer (64-bit)	Windows	Recommended	27.7 MB	.sigstore	SPDX	SIG	96159fcb523ae404b707186a75b4104ee23851e476a5e838e14584cf1e03f981
Windows installer (32-bit)	Windows		26.5 MB	.sigstore	SPDX	SIG	9203d78e635e4f348d62f574a01cc58c

Step 5: Here you see a different version of python along with the operating system

Version	Operating system	Description	File size	Sigstore	SBOM	GPG	SHA-256 checksum
Gzipped source tarball	Source release		28.4 MB	.sigstore	SPDX	SIG	12e7cb170ad2d1a69aee96a1cc7fc8de5b1e97a2bdac51683a3db016ec9a2996
XZ compressed source tarball	Source release		21.9 MB	.sigstore	SPDX	SIG	2a84cd31dd8d8ea8aaff75de66fc1b4b0127dd5799aa50a64ae9a313885b4593
macOS installer	macOS	for macOS 10.13 and later	67.9 MB	.sigstore		SIG	b8d2eb5b6eabf583dce703027f6d4ed4b71c9bd1b05ad35d1f0c23f78e9aea76
Windows installer (64-bit)	Windows	Recommended	27.7 MB	.sigstore	SPDX	SIG	96159fcb523ae404b707186a75b4104ee23851e476a5e838e14584cf1e03f981
Windows installer (32-bit)	Windows		26.5 MB	.sigstore	SPDX	SIG	9203d78e635e4f348d62f574a01cc58c7a1ba87d30252f5e0e1f5c0330667a6e
Windows installer (ARM64)	Windows	Experimental	27.1 MB	.sigstore	SPDX	SIG	e4476454abcc329b04d330a296995cce5530544d3d2fc006d80f17ae0437fb8c
Windows embeddable package (64-bit)	Windows		10.4 MB	.sigstore	SPDX	SIG	76f238f606250c87c6beac75dccd35ee99070a13490555936abb0cb64ecce3d8
Windows embeddable package (32-bit)	Windows		9.3 MB	.sigstore	SPDX	SIG	51ec4c741212f9488d070033bd2c8a67fc27cf7d36abffee7cb0f4481c9fb52b
Windows embeddable package (ARM64)	Windows		9.9 MB	.sigstore	SPDX	SIG	b034042d46e20de57dab22de1813d56bde286899d9d9991255a7c0e1f7efbfa0
Windows release manifest	Windows	Install with 'py install 3.13'	14.6 KB	.sigstore			7674afb28fcf4acb4866b3f8d010929f92f725e62b0d437330d11228b44a1823

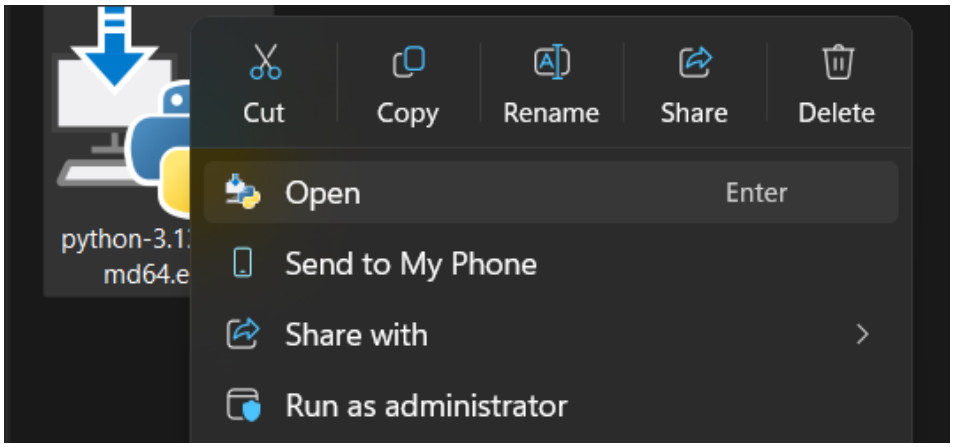
To download Windows 32-bit python, you can select any one from the three options: Windows x86 embeddable zip file, Windows x86 executable installer or Windows x86 web-based installer.

To download Windows 64-bit python, you can select any one from the three options: Windows x86-64 embeddable zip file, Windows x86-64 executable installer or Windows x86-64 web-based installer.

Here we will install Windows x86-64 web-based installer. Here your first part regarding which version of python is to be downloaded is completed. Now we move ahead with the second part in installing python i.e. Installation.

Installation of Python

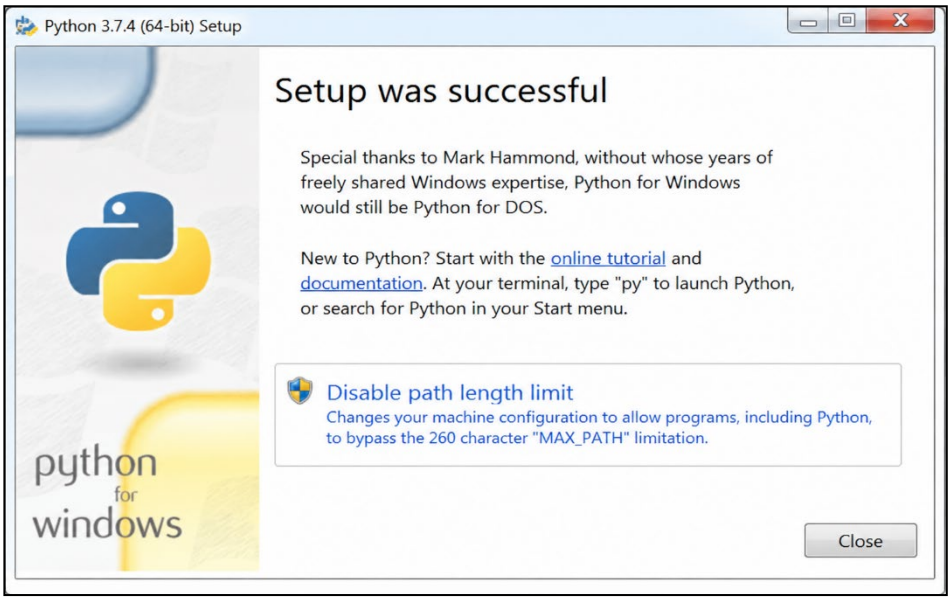
Step 1: Go to Download and Open the downloaded python version to carry out the installation process.



Step 2: Before you click on Install Now, make sure to tick on Add Python to PATH.



Step 3: Click on Install NOW After the installation is successful. Click on Close.

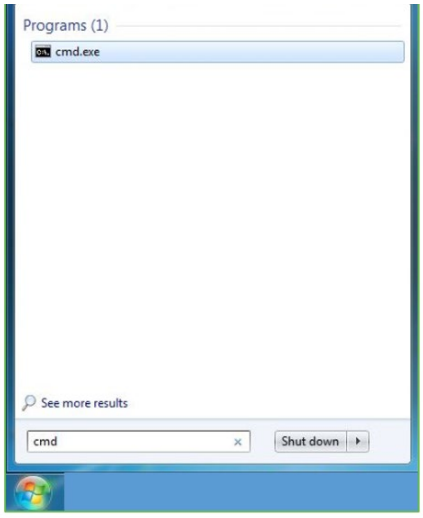


Note: The installation process might take a couple of minutes.

Verify the Python Installation

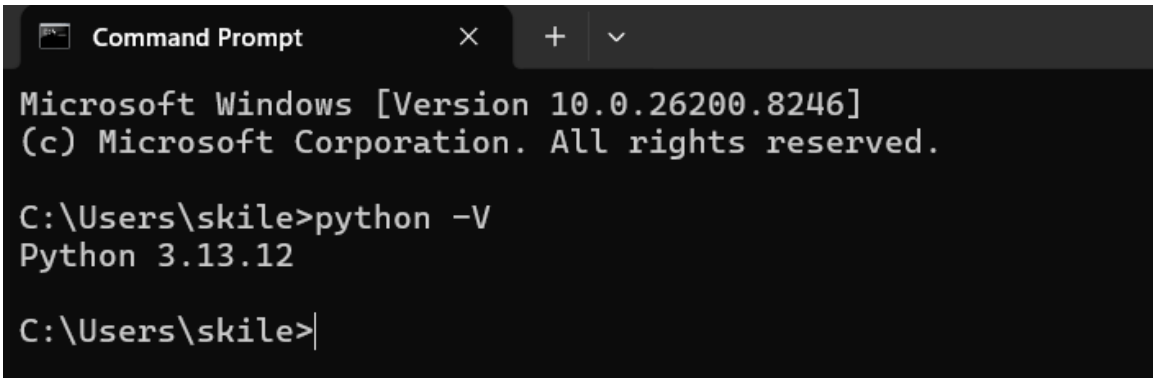
Step 1: Click on Start

Step 2: In the Windows Run Command, type “cmd”.



Step 3: Open the Command prompt option.

Step 4: The python is correctly installed. Type **python –V** and press Enter.

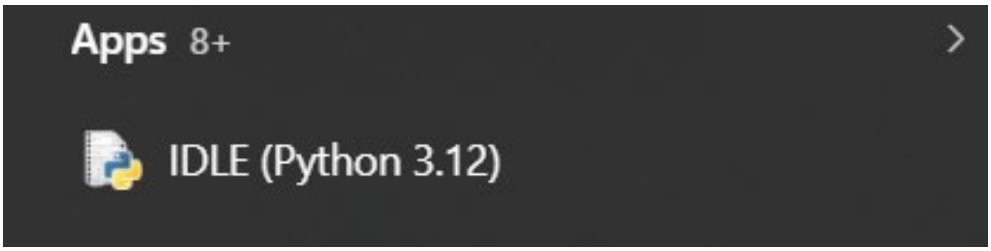


Step 5: You will get the answer as 3.13.12

Note: If you have any of the earlier versions of Python already installed. You must first uninstall the earlier version and then install the new one.

Step 1: Click on Start

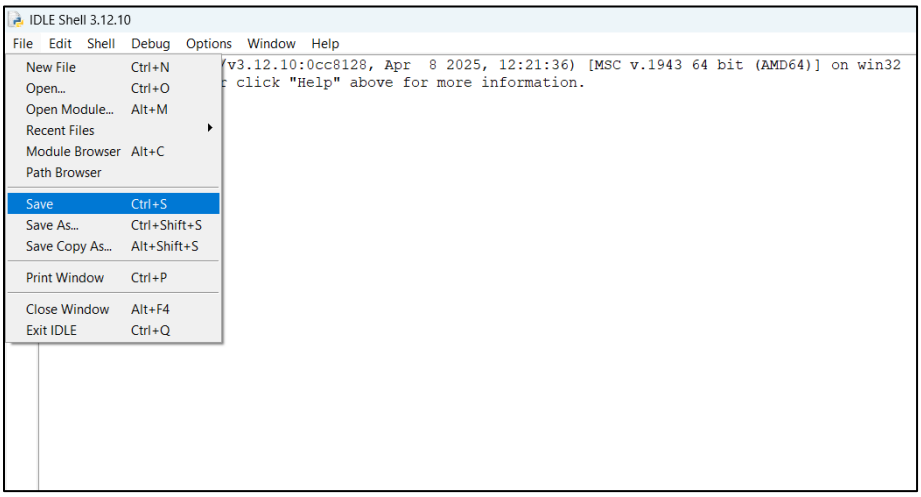
Step 2: In the Windows Run command, type “python idle”.



Step 3: Click on IDLE (Python 3.13.12 64-bit) and launch the program.

Step 4: To go ahead with working in IDLE you must first save the file.

Click on File > Click on Save



Step 5: Name the file and save as type should be Python files. Click on SAVE. Here I have named the files as Hey World.

Step 6: Now for e.g. **enter print**

SOURCE CODE

Here we use this source code to run our “Student Portal System,” a web-based application developed using Python, Django, and WampServer (MySQL). It uses machine learning techniques like Naive Bayes and XGBoost for student performance prediction. The system includes features like login, marks analysis, file upload/download, attendance, and feedback.

1. url.py

2.views.py

These are the two important source code files to run the entire web-based application

1. url.py

```
from django.urls import path
from . import views
urlpatterns = [
    # ROOT URL
    path("", views.index, name="home"),
    path("index/", views.index, name="index"),

    # Admin URLs
    path("admin-login/", views.AdminLogin, name="AdminLogin"),
    path("admin-login-action/", views.AdminLoginAction, name="AdminLoginAction"),
```

```
# Faculty URLs
  path("add-faculty/", views.AddFaculty, name="AddFaculty"),
  path("add-faculty-action/", views.AddFacultyAction, name="AddFacultyAction"),
  path("faculty-login/", views.FacultyLogin, name="FacultyLogin"),
  path("faculty-login-action/", views.FacultyLoginAction, name="FacultyLoginAction"),
  path("view-faculty/", views.ViewFaculty, name="ViewFaculty"),
  path("choose-faculty/", views.ChooseFaculty, name="ChooseFaculty"),
  path("faculty-suggestion/", views.FacultySuggestion, name="FacultySuggestion"),
  path("auto-faculty-selection/", views.AutoFacultySelection, name="AutoFacultySelection"),
  # Student URLs
  path("add-student/", views.AddStudent, name="AddStudent"),
  path("add-student-action/", views.AddStudentAction, name="AddStudentAction"),
  path("student-login/", views.StudentLogin, name="StudentLogin"),
  path("student-login-action/", views.StudentLoginAction, name="StudentLoginAction"),
  path("view-student/", views.ViewStudent, name="ViewStudent"),
  path("student-screen/", views.StudentScreen, name="StudentScreen"),
  # Academics & Performance URLs
  path("school-performance/", views.SchoolPerformance, name="SchoolPerformance"),
  path("add-attendance/", views.AddAttendance, name="AddAttendance"),
  path("add-attendance-action/", views.AddAttendanceAction, name="AddAttendanceAction"),
  path("add-marks/", views.AddMarks, name="AddMarks"),
  path("add-marks-action/", views.AddMarksAction, name="AddMarksAction"),
  path("view-marks/", views.ViewMarks, name="ViewMarks"),
  path("view-marks-action/", views.ViewMarksAction, name="ViewMarksAction"),
  path("view-progress-report/", views.ViewProgressReport, name="ViewProgressReport"),
  path("view-progress-report-action/", views.ViewProgressReportAction, name="ViewProgressReportAction"),
```

```
# Materials & Assignments URLs
    path("create-assignments/", views.CreateAssignments, name="CreateAssignments"),
    path("create-assignments-action/", views.CreateAssignmentsAction, name="CreateAssignmentsAction"),
    path("upload-material/", views.UploadMaterial, name="UploadMaterial"),
    path("upload-material-action/", views.UploadMaterialAction, name="UploadMaterialAction"),
    path("download-materials/", views.DownloadMaterials, name="DownloadMaterials"),
    path("download-material-action/", views.DownloadMaterialAction, name="DownloadMaterialAction"),
    path("view-assignments/", views.ViewAssignments, name="ViewAssignments"),

    # Messaging & Feedback URLs
    path("student-messaging/", views.StudentMessaging, name="StudentMessaging"),
    path("student-messaging-action/", views.StudentMessagingAction, name="StudentMessagingAction"),
    path("messaging/", views.Messaging, name="Messaging"),
    path("messaging-action/", views.MessagingAction, name="MessagingAction"),
    path("view-messages/", views.ViewMessages, name="ViewMessages"),
    path("view-student-messages/", views.ViewStudentMessages, name="ViewStudentMessages"),
    path("feedback/", views.Feedback, name="Feedback"),
    path("feedback-action/", views.FeedbackAction, name="FeedbackAction"),
]
```

2. view.py:

```
import os
import io
import base64
from datetime import datetime

from django.shortcuts import render
from django.http import HttpResponse
from django.views.decorators.csrf import csrf_exempt

import pymysql
import numpy as np
import pandas as pd

from sklearn.preprocessing import LabelEncoder, MinMaxScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.naive_bayes import GaussianNB
from xgboost import XGBClassifier

import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt


# ======================================================
# GLOBAL CONFIG
# ======================================================

UPLOAD_DIR = "StudentApp/static/files/"

graph = ""
accuracy = []
precision = []
recall = []
fscore = []
```

```
# ============================================================
# DATABASE CONNECTION
# ============================================================
def get_db_connection():
return pymysql.connect(
host='127.0.0.1',
user='root',
password='',
database='portal_db',
charset='utf8'
)
# ============================================================
# MACHINE LEARNING SETUP
# ============================================================
dataset = pd.read_csv("model/dataset.csv")
le = LabelEncoder()
dataset['FinalResult'] = le.fit_transform(dataset['FinalResult'].astype(str))
labels = le.classes_

Y = dataset['FinalResult'].to_numpy()
X = dataset.drop(columns=['FinalResult']).to_numpy()

sc = MinMaxScaler()
X = sc.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
X, Y, test_size=0.2, random_state=42
)
nb_cls = GaussianNB()
nb_cls.fit(X_train, y_train)
xg_cls = XGBClassifier(use_label_encoder=False, eval_metric='mlogloss')
xg_cls.fit(X_train, y_train)

def calculateMetrics(y_true, y_pred):
accuracy.append(round(accuracy_score(y_true, y_pred) * 100, 2))
precision.append(round(precision_score(y_true, y_pred, average='macro', zero_division=0) * 100, 2))
recall.append(round(recall_score(y_true, y_pred, average='macro', zero_division=0) * 100, 2))
fscore.append(round(f1_score(y_true, y_pred, average='macro', zero_division=0) * 100, 2))
```

```
def runML(value):
try:
feature_count = sc.n_features_in_
data = [value] + [0.0] * (feature_count - 1)
data = np.array([data])
data = sc.transform(data)
pred = xg_cls.predict(data)[0]
return labels[pred]
except:
return "Error"
# ============================================================
# BASIC VIEWS
# ============================================================
def index(request):
return render(request, "index.html")
# ============================================================
# FILE UPLOAD / DOWNLOAD
# ============================================================

def upload_page(request):
return render(request, "upload.html")


@csrf_exempt
def upload_file(request):
if request.method == "POST":
file = request.FILES.get("file")

if file:
os.makedirs(UPLOAD_DIR, exist_ok=True)
file_path = os.path.join(UPLOAD_DIR, file.name)

with open(file_path, "wb+") as destination:
for chunk in file.chunks():
destination.write(chunk)

return HttpResponse("File Uploaded Successfully")

return HttpResponse("Upload Failed")
```

```
def download_file(request):
filename = request.GET.get("name")
file_path = os.path.join(UPLOAD_DIR, filename)
if os.path.exists(file_path):
with open(file_path, "rb") as f:
response = HttpResponse(f.read(), content_type="application/octet-stream")
response["Content-Disposition"] = f'attachment; filename="{filename}"'
return response
return HttpResponse("File Not Found")
# ============================================================
# STUDENT MARKS + ML
# ============================================================
def view_marks(request):
if request.method == "POST":
uname = request.POST.get("username")
course = request.POST.get("course")
year = request.POST.get("year")
con = get_db_connection()
total = 0
count = 0
output = "<table border=1>"
with con:
cur = con.cursor()
cur.execute(
"SELECT subject_name, subject_marks FROM marks WHERE student_name=%s AND course_year=%s AND
course_name=%s",
(uname, year, course))
rows = cur.fetchall()
for row in rows:
output += f"<tr><td>{row[0]}</td><td>{row[1]}</td></tr>"
total += float(row[1])
count += 1
avg = total / count if count else 0
prediction = runML(avg)
output += f"<tr><td>Total</td><td>{total}</td></tr>"
output += f"<tr><td>Average</td><td>{avg}</td></tr>"
output += f"<tr><td>Prediction</td><td>{prediction}</td></tr>"
output += "</table>"
return HttpResponse(output)
return render(request, "marks.html")
```

```
avg = total / count if count else 0
prediction = runML(avg)

output += f"<tr><td>Total</td><td>{total}</td></tr>"
output += f"<tr><td>Average</td><td>{avg}</td></tr>"
output += f"<tr><td>Prediction</td><td>{prediction}</td></tr>"
output += "</table>"

return HttpResponse(output)
return render(request, "marks.html")
# ============================================================
# GRAPH GENERATION
# ============================================================
def performance_graph(request):
scores = [80, 70, 90]
labels = ["Math", "Science", "English"]
plt.figure(figsize=(5, 5))
plt.pie(scores, labels=labels, autopct="%1.1f%%")
buf = io.BytesIO()
plt.savefig(buf, format="png")
plt.close()

img = base64.b64encode(buf.getvalue()).decode()
return render(request, "graph.html", {"img": img})
# ============================================================
# SIMPLE LOGIN SYSTEM
# ============================================================
def login(request):
return render(request, "login.html")

def login_action(request):
if request.method == "POST":
user = request.POST.get("username")
pwd = request.POST.get("password")

if user == "admin" and pwd == "admin":
return HttpResponse("Login Success")
else:
return HttpResponse("Login Failed")
```

```
# ============================================================
# DATABASE INSERT EXAMPLE
# ============================================================
def add_student(request):
if request.method == "POST":
name = request.POST.get("name")
course = request.POST.get("course")

con = get_db_connection()

with con:
cur = con.cursor()
cur.execute(
"INSERT INTO student(student_name, course_name) VALUES(%s,%s)",
(name, course)
)
con.commit()

return HttpResponse("Student Added")

return render(request, "add_student.html")
# ============================================================
# MESSAGE SYSTEM
# ============================================================
def send_message(request):
if request.method == "POST":
sender = request.POST.get("sender")
receiver = request.POST.get("receiver")
msg = request.POST.get("message")

con = get_db_connection()

with con:
cur = con.cursor()
cur.execute(
"INSERT INTO messages VALUES(%s,%s,%s,%s)",
(sender, receiver, msg, datetime.now())
)
con.commit()
return HttpResponse("Message Sent")

return render(request, "message.html")
```

```
# ============================================================
# VIEW MESSAGES
# ============================================================

def view_messages(request):
con = get_db_connection()
output = "<table border=1>"

with con:
cur = con.cursor()
cur.execute("SELECT * FROM messages")
rows = cur.fetchall()

for row in rows:
output += f"<tr><td>{row[0]}</td><td>{row[1]}</td><td>{row[2]}</td></tr>"

output += "</table>"
return HttpResponse(output)

# ============================================================
# FACULTY MANAGEMENT
# ============================================================
def add_faculty(request):
if request.method == "POST":
name = request.POST.get("name")
subject = request.POST.get("subject")
username = request.POST.get("username")
password = request.POST.get("password")
con = get_db_connection()
with con:
cur = con.cursor()
cur.execute(
"INSERT INTO faculty(faculty_name, teaching_subjects, username, password) VALUES(%s,%s,%s,%s)",
(name, subject, username, password)
)
con.commit()
```

```
return HttpResponse("Faculty Added")
return render(request, "add_faculty.html")

def view_faculty(request):
con = get_db_connection()
output = "<table border=1>"

with con:
cur = con.cursor()
cur.execute("SELECT faculty_name, teaching_subjects FROM faculty")
rows = cur.fetchall()

for row in rows:
output += f"<tr><td>{row[0]}</td><td>{row[1]}</td></tr>"

output += "</table>"
return HttpResponse(output)

# ============================================================
# ATTENDANCE SYSTEM
# ============================================================
def mark_attendance(request):
if request.method == "POST":
student = request.POST.get("student")
faculty = request.POST.get("faculty")
date = datetime.now().strftime("%Y-%m-%d")

con = get_db_connection()
with con:
cur = con.cursor()
cur.execute(
"INSERT INTO student_attendance VALUES(%s,%s,%s)",
(student, faculty, date)
)
con.commit()

return HttpResponse("Attendance Marked")
return render(request, "attendance.html")
```

```
def view_attendance(request):
con = get_db_connection()
output = "<table border=1>"

with con:
cur = con.cursor()
cur.execute("SELECT * FROM student_attendance")
rows = cur.fetchall()

for row in rows:
output += f"<tr><td>{row[0]}</td><td>{row[1]}</td><td>{row[2]}</td></tr>"

output += "</table>"
return HttpResponse(output)

# ============================================================
# ASSIGNMENT SYSTEM
# ============================================================
def create_assignment(request):
if request.method == "POST":
faculty = request.POST.get("faculty")
subject = request.POST.get("subject")
task = request.POST.get("task")
desc = request.POST.get("desc")
date = request.POST.get("date")
con = get_db_connection()
with con:
cur = con.cursor()
cur.execute(
"INSERT INTO assignments VALUES(%s,%s,%s,%s,%s)",
(faculty, subject, task, desc, date)
)
con.commit()

return HttpResponse("Assignment Created")

return render(request, "assignment.html")
```

```
def view_assignments(request):
con = get_db_connection()
output = "<table border=1>"

with con:
cur = con.cursor()
cur.execute("SELECT * FROM assignments")
rows = cur.fetchall()

for row in rows:
output += f"<tr><td>{row[0]}</td><td>{row[1]}</td><td>{row[2]}</td></tr>"

output += "</table>"
return HttpResponse(output)

# ============================================================
# FEEDBACK SYSTEM
# ============================================================
def submit_feedback(request):
if request.method == "POST":
student = request.POST.get("student")
faculty = request.POST.get("faculty")
feedback = request.POST.get("feedback")
rating = request.POST.get("rating")
con = get_db_connection()
with con:
cur = con.cursor()
cur.execute(
"INSERT INTO feedback VALUES(%s,%s,%s,%s)",
(student, faculty, feedback, rating)
)
con.commit()

return HttpResponse("Feedback Submitted")

return render(request, "feedback.html")
```

```
def view_feedback(request):
con = get_db_connection()
output = "<table border=1>"

with con:
cur = con.cursor()
cur.execute("SELECT * FROM feedback")
rows = cur.fetchall()

for row in rows:
output += f"<tr><td>{row[0]}</td><td>{row[1]}</td><td>{row[2]}</td></tr>"

output += "</table>"
return HttpResponse(output)

# ============================================================
# FILE LIST + DELETE SYSTEM
# ============================================================
def list_files(request):
os.makedirs(UPLOAD_DIR, exist_ok=True)
files = os.listdir(UPLOAD_DIR)

output = "<h3>Files</h3><ul>"
for f in files:
output += f'<li>{f} - <a href="/download/?name={f}">Download</a> - <a
href="/delete/?name={f}">Delete</a></li>'
output += "</ul>"

return HttpResponse(output)
def delete_file(request):
filename = request.GET.get("name")
file_path = os.path.join(UPLOAD_DIR, filename)
if os.path.exists(file_path):
os.remove(file_path)
return HttpResponse("File Deleted")
return HttpResponse("File Not Found")
```

```
# ========================================================
# SIMPLE DASHBOARD STATS
# ========================================================

def dashboard(request):
con = get_db_connection()

stats = {
"students": 0,
"faculty": 0,
"assignments": 0
}

with con:
cur = con.cursor()

cur.execute("SELECT COUNT(*) FROM student")
stats["students"] = cur.fetchone()[0]

cur.execute("SELECT COUNT(*) FROM faculty")
stats["faculty"] = cur.fetchone()[0]

cur.execute("SELECT COUNT(*) FROM assignments")
stats["assignments"] = cur.fetchone()[0]

return render(request, "dashboard.html", {"stats": stats})
```

SYSTEM TESTING

System Testing is an important phase in software development used to identify errors and verify whether the Student Portal System functions correctly according to the project requirements. Testing ensures that all modules such as login, attendance management, assignment handling, marks entry, study material management, communication, and Machine Learning prediction work properly without failure.

The main purpose of testing is to discover faults, improve system reliability, and ensure that the application meets user expectations. It helps in checking the functionality, performance, security, and accuracy of the system. Different testing methods are performed to validate individual modules as well as the complete integrated system.

The Student Portal System undergoes several types of testing such as Unit Testing, Integration Testing, Functional Testing, System Testing, and User Acceptance Testing. These testing methods ensure that the system operates efficiently, securely, and reliably for Admins, Teachers, and Students.

Testing Methods

Unit Testing

Unit Testing is used to verify whether individual modules of the Student Portal System work correctly. It checks modules such as login, attendance, assignments, marks entry, and Machine Learning prediction before integrating them with other modules.

Integration Testing

Integration Testing verifies whether different modules of the system work together properly. It checks the interaction between Admin, Teacher, Student, Database, and Machine Learning modules to ensure smooth data flow.

Functional Testing

Functional Testing ensures that all features of the Student Portal System function according to the project requirements. It tests modules such as login, attendance management, assignments, marks entry, communication, and performance prediction.

System Testing

System Testing evaluates the complete Student Portal System to ensure that the application works correctly, securely, and reliably as a whole system.

White Box Testing

White Box Testing is performed with knowledge of the internal code structure. It is used to test program logic, conditions, loops, and code execution within the application.

Black Box Testing

Black Box Testing is performed without knowledge of the internal code. The tester verifies system functionality by checking inputs and outputs based on user requirements.

Test Strategy and Approach

Testing is performed manually to verify that all modules function correctly and provide accurate results.

Acceptance Testing

Acceptance Testing is performed to ensure that the Student Portal System satisfies user requirements and works effectively for Admins, Teachers, and Students.

Test Results

All test cases of the Student Portal System were executed successfully without major defects or errors.

Test Cases

Test Cases are designed to verify whether the Student Portal System functions correctly according to the specified project requirements. Different test cases are performed for modules such as user login, attendance management, assignment handling, marks entry, study material upload and download, messaging, and Machine Learning based student performance prediction.

The testing process checks whether all modules provide accurate outputs, proper input validation, secure data handling, smooth navigation, and reliable system performance. Each functionality of the system is tested carefully to ensure efficient operation for Admins, Teachers, and Students. All test cases were executed successfully, and no major defects or errors were identified during testing.

Test cases1: -

Test case for Login form:

FUNCTION:	**LOGIN**
EXPECTED RESULTS:	Should Validate the user and check his existence in database
ACTUAL RESULTS:	Validate the user and checking the user against the database
LOW PRIORITY	**No**
HIGH PRIORITY	**Yes**

Test case2:

Test case for User Registration form:

FUNCTION:	**USER REGISTRATION**
EXPECTED RESULTS:	Should check if all the fields are filled by the user and saving the user to database.
ACTUAL RESULTS:	Checking whether all the fields are field by user or not through validations and saving user.
LOW PRIORITY	**No**
HIGH PRIORITY	**Yes**

Test case3:

Test case for Change Password:

When the old password does not match with the new password, then this results in displaying an error message as "OLD PASSWORD DOES NOT MATCH WITH THE NEW PASSWORD".

FUNCTION:	**Change Password**
EXPECTED RESULTS:	Should check if old password and new password fields are filled by the user and saving the user to database.
ACTUAL RESULTS:	Checking whether all the fields are field by user or not through validations and saving user.
LOW PRIORITY	**No**
HIGH PRIORITY	**Yes**

RESULTS

The Student Portal System performs efficiently under different testing conditions. All modules such as login, attendance management, assignment handling, marks entry, study material upload, communication, and student performance prediction were tested successfully.

The system generated accurate outputs for valid inputs and handled invalid inputs properly. The testing results confirm that the system is reliable, user-friendly, secure, and satisfies the specified project requirements without major defects.

In this project, Machine Learning algorithms such as Naïve Bayes and XGBoost are used to predict student performance based on historical academic data. The system analyses student marks and generates.

performance feedback such as “Good,” “Fair,” and “Poor.” Among the algorithms used, XGBoost achieved more than 93% prediction accuracy. to implement this project, the following modules were developed.

1) Admin Module

The admin can login to the system and manage teacher details, student records, subjects, timetables, notifications, and academic reports. The admin also monitors overall system activities.

2) Teacher Module

Teachers can login to the system and manage attendance, upload study materials, create assignments, enter marks, and communicate with students. Teachers can also view student performance predictions generated by the Machine Learning module.

3) Student Module

Students can login to the portal and view attendance, assignments, marks, study materials, and performance feedback. Students can also communicate with teachers through the messaging module.

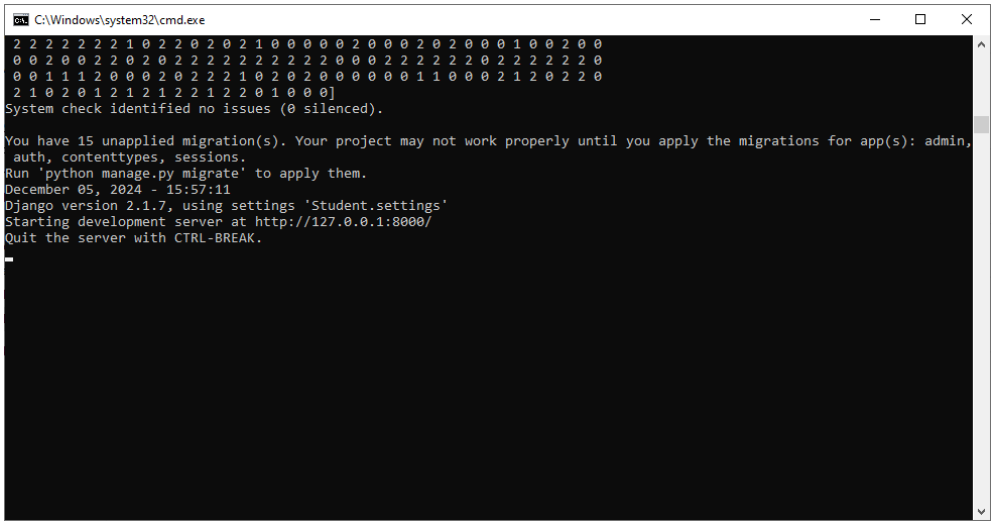
4) Machine Learning Module

The Machine Learning module processes student academic data and predicts student performance using Naïve Bayes and XGBoost algorithms. The prediction results help teachers identify weak students and provide academic support.

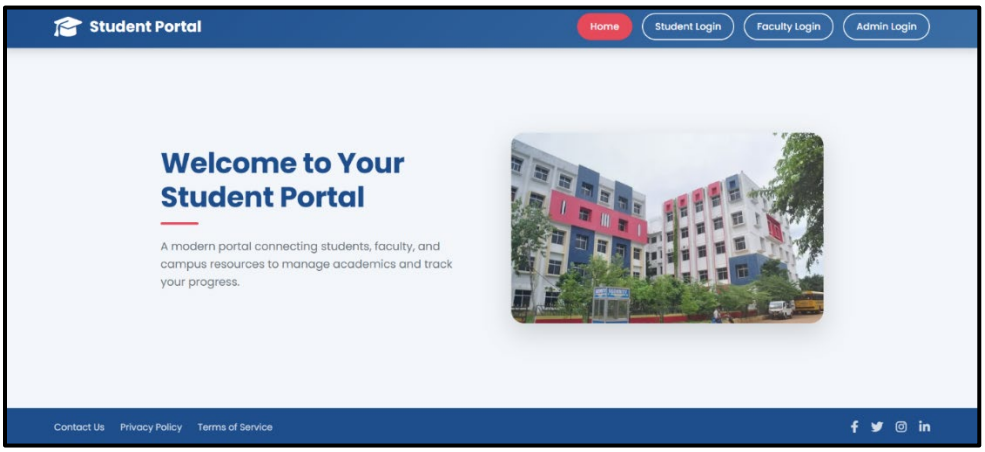
To run the project, install MySQL database and create the required database tables. Install Python 3.13.12 and required packages from the requirements file. After running the Django server, the Student Portal System can be accessed through a web browser.

To run project, install MYSQL database and then open MYSQL console and then copy content from database.txt and paste in MYSQL console to create database

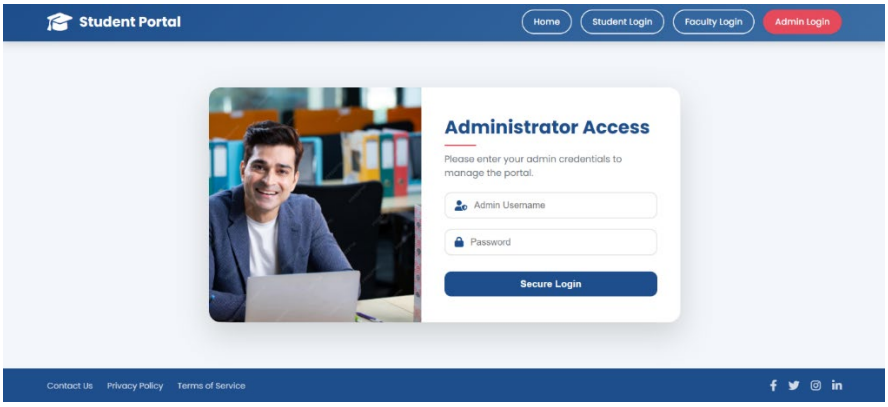
Install python 3.13.12 and then install all packages given in requirements.txt file and then double click on 'run.bat' file to start python web server and will get below page



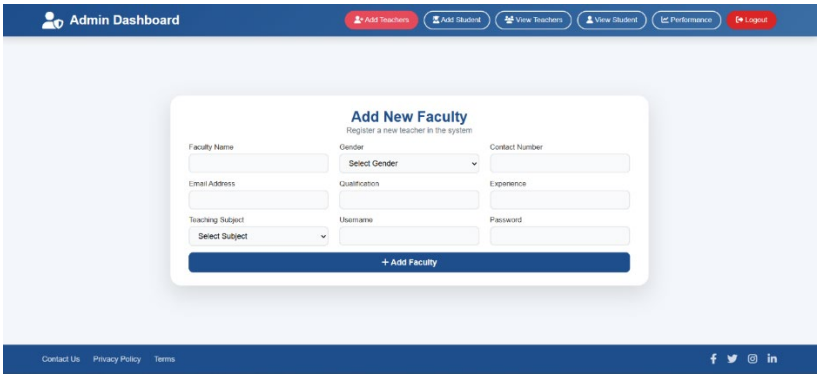
In above screen python web server started and now open browser and enter URL as http://127.0.0.1:8000/index.html and press enter key to get below page



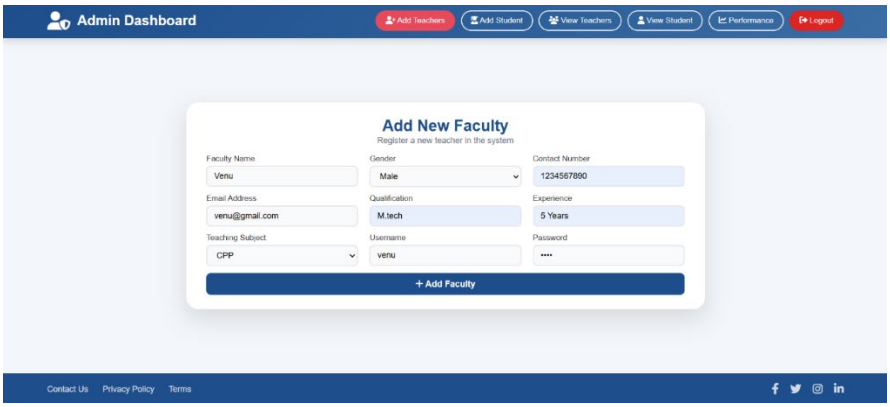
In above screen can see home page of the application and can see performance of ML algorithm showing in tabular format and in both algorithms XGBOOST got high accuracy. Now click on Admin Login' link to get below page.



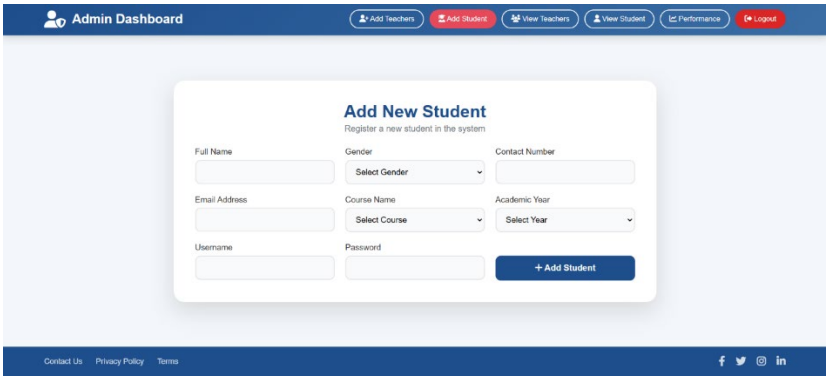
In above screen admin is login and after login will get below page



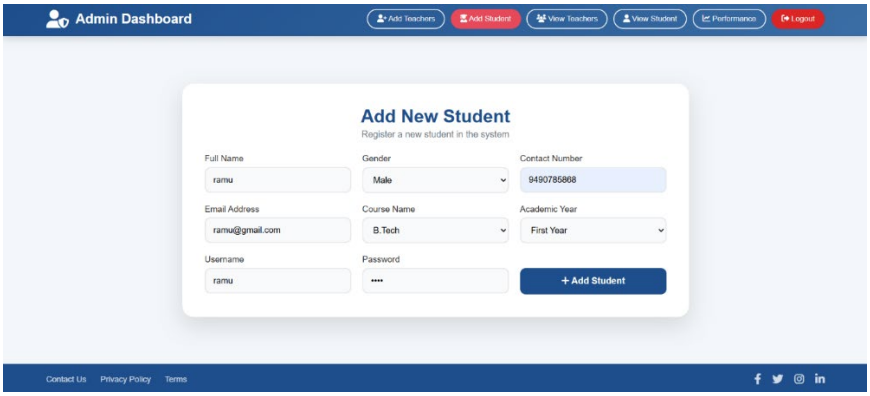
In above screen admin can click on 'Add Teacher' link to add new faculty details



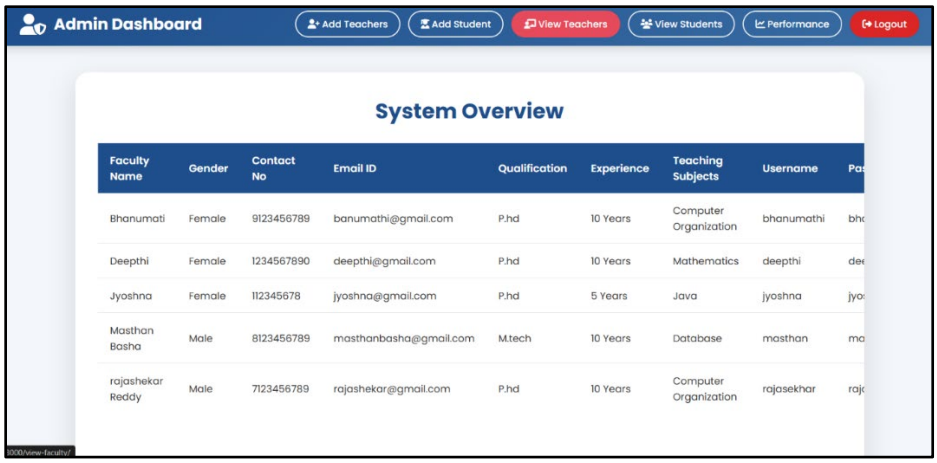
In above screen admin is adding faculty details then press button to get below page



In above screen faculty details added and now click on ‘Add Student’ details to get below page

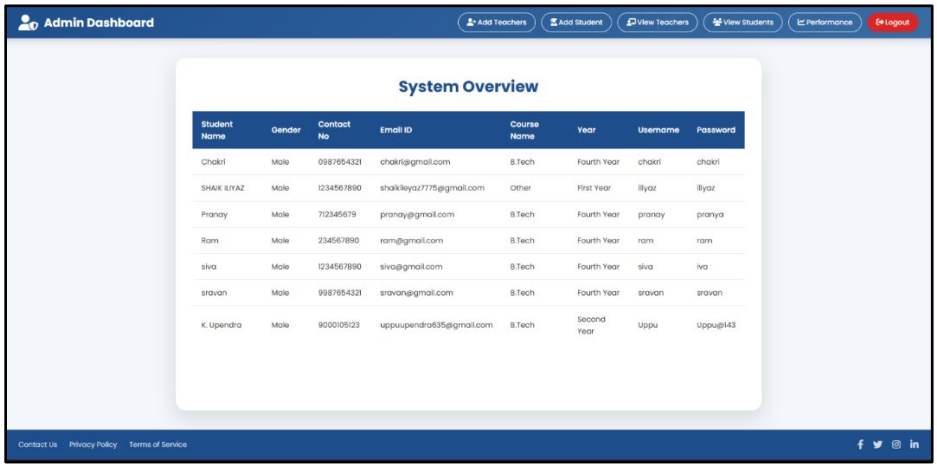


In above screen adding student details and then press button to get below page and give valid email

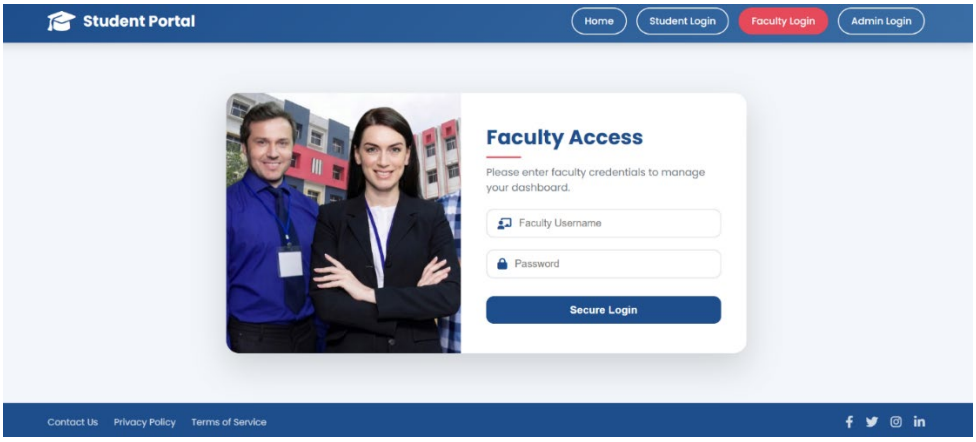


In above screen student details added and similarly you can add any number of students and now click on 'View Teachers' link to view list of existing faculty details

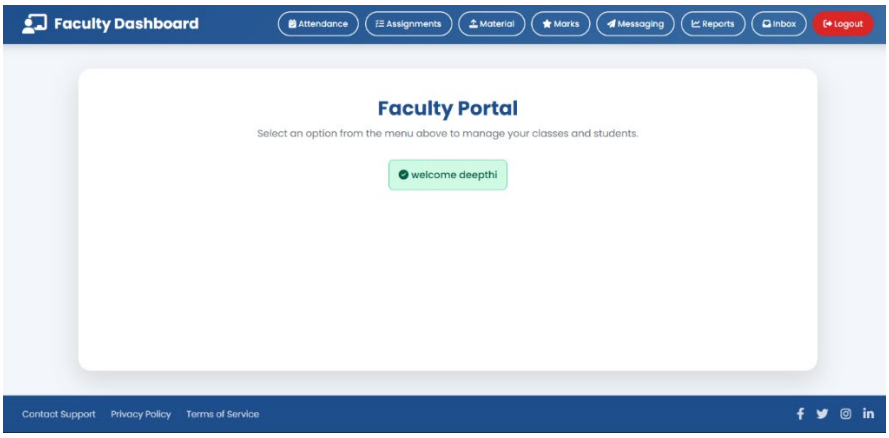
In above screen admin can view list of faculties and now click on 'View Student' link to get list of available students



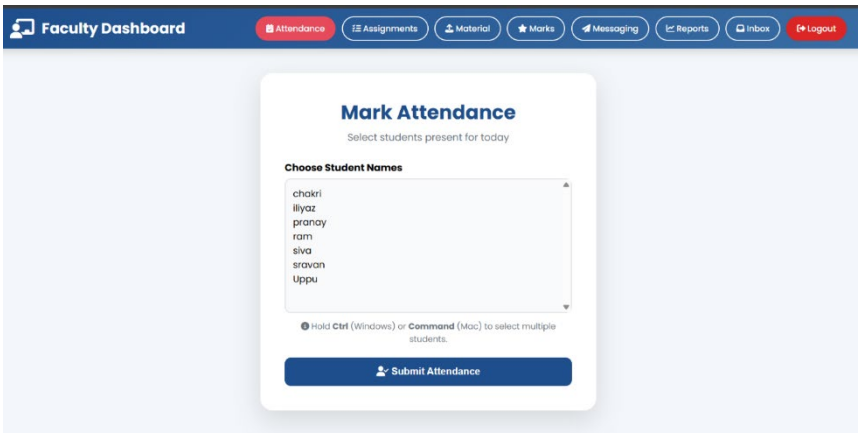
In above screen can view list of available students and can view performance after adding marks and now logout and login as Faculty



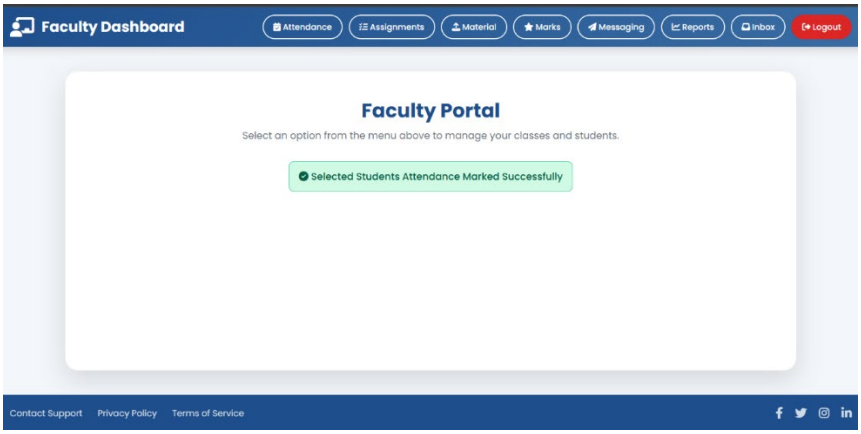
In above screen faculty is login and after login will get below page



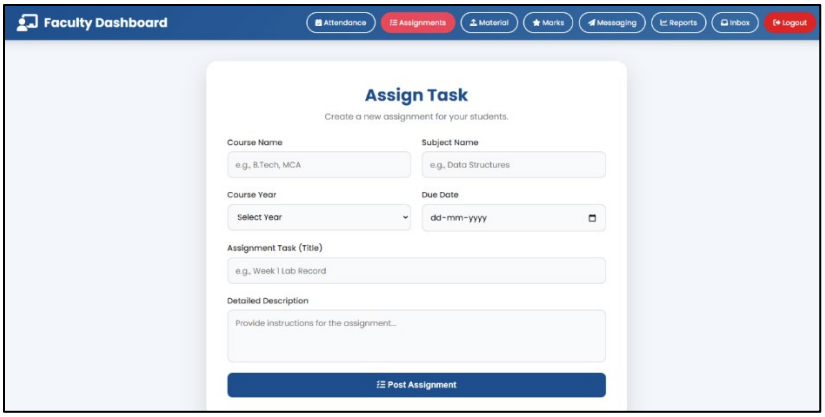
In above screen faculty can click on ‘Add Attendance’ link to get list of available students like below page



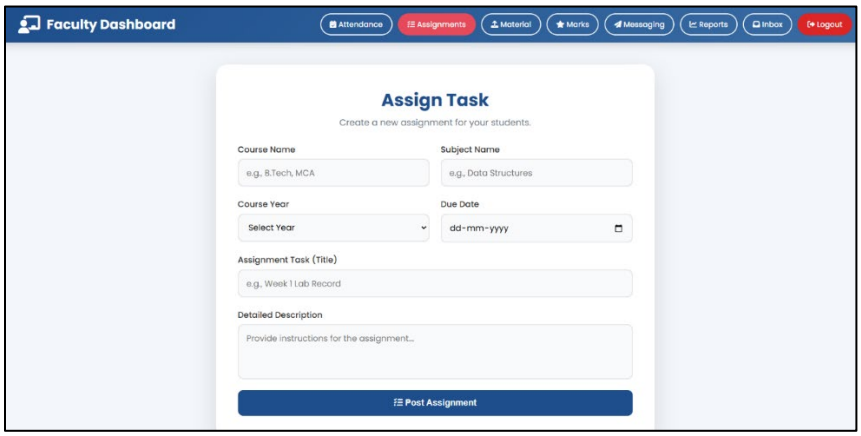
In above screen faculty can hold CTRL key and then select all students' names and then press button to mark today's attendance and get below page



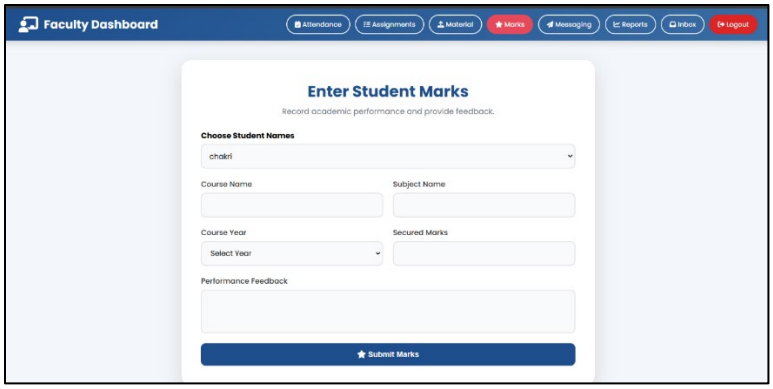
In above screen attendance marked successfully and now click on 'Create Assignments' link to get below page



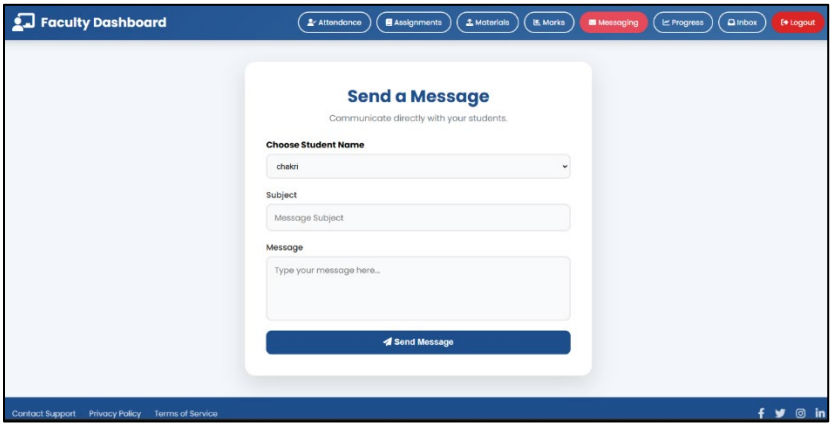
In above screen faculty will create new task or assignment and then press button to get below page



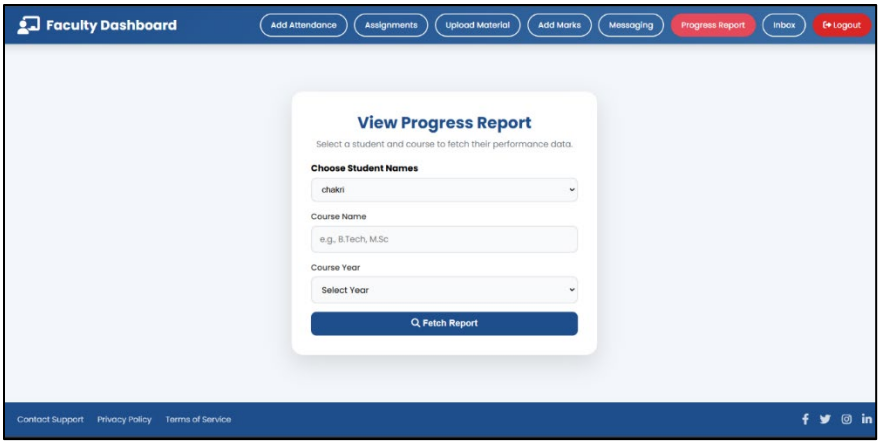
In above screen assignment details added and now click on ‘Add Marks’ link to get below page



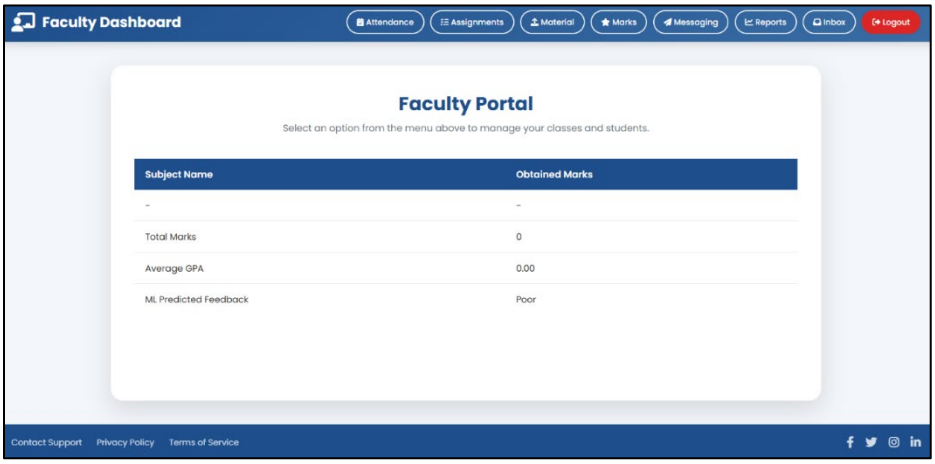
In above screen faculty awarding marks to student on specific subject and now click on ‘Messaging’ link to send email to parents about student activities



Using above module faculty will send email to student parents about his performance and now click on ‘View Progress Report’ link to get below page

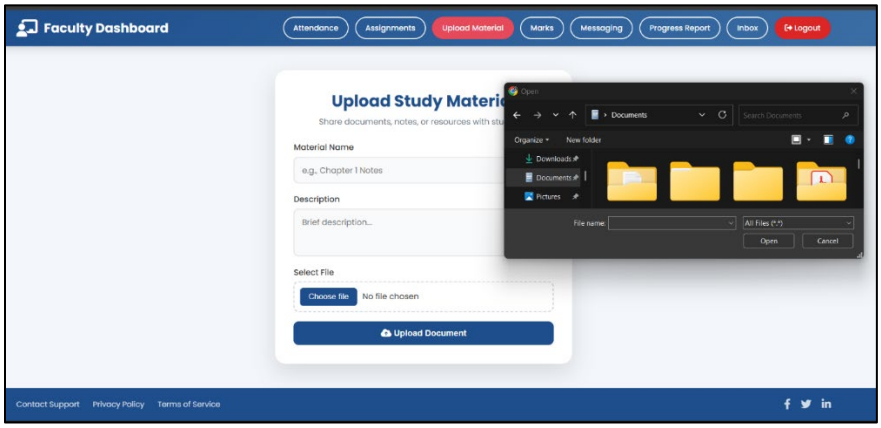


In above screen faculty can select student name and then choose course name and year and then press button to get below page

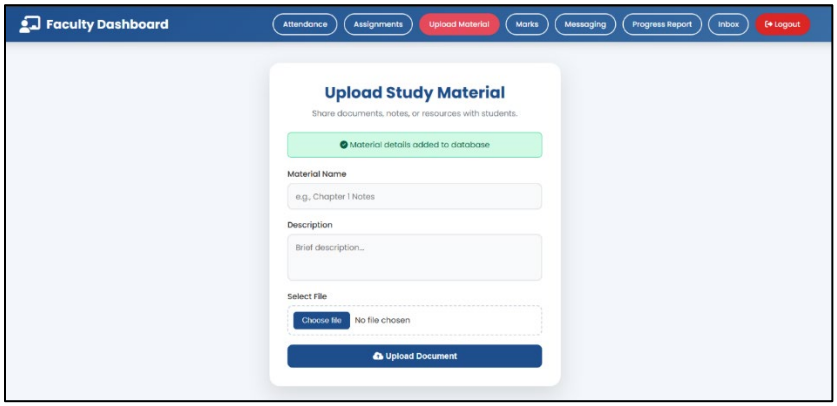


In above screen faculty can see students' subject wise marks and then can see total and average marks and then ML can predict performance of student as 'Good, Fair or Poor' and in above screen as per student performance ML has predicted feedback as 'Good' and now select another student and see ML feedback in above screen selected another student as 'Dave' and then press button

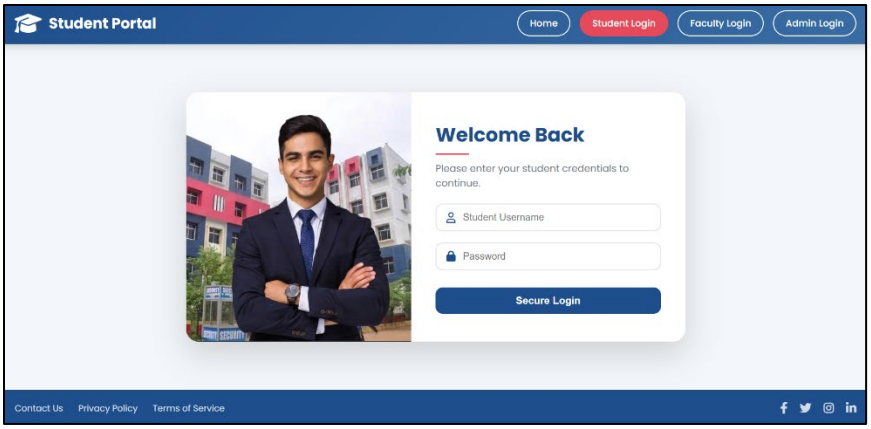
In above screen based on student performance ML has predicted feedback ‘Fair’ and now click on ‘View Student Messages’ link to view student’s messages In above screen faculty will see students and will clear doubt by sending message to student and now click on ‘Upload Material’ link to upload material and get below page



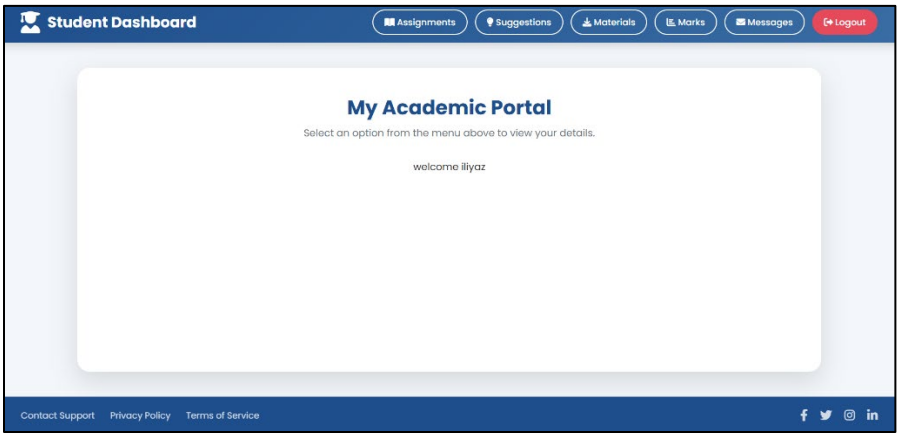
In above screen faculty is uploading some material and now click ‘Open and Submit ‘to get



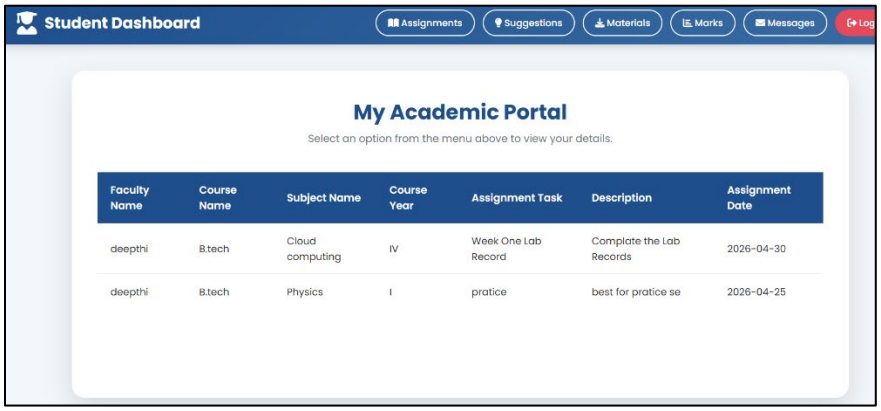
In above screen material successfully uploaded and now logout and login as student



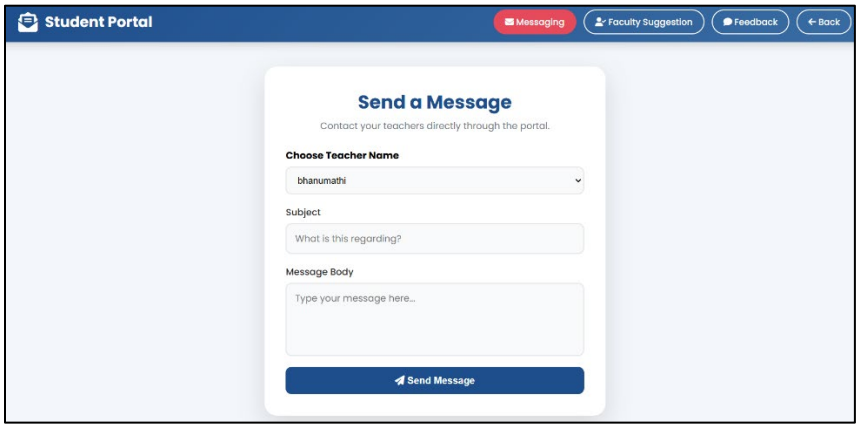
In above screen student is login and after login will get below page



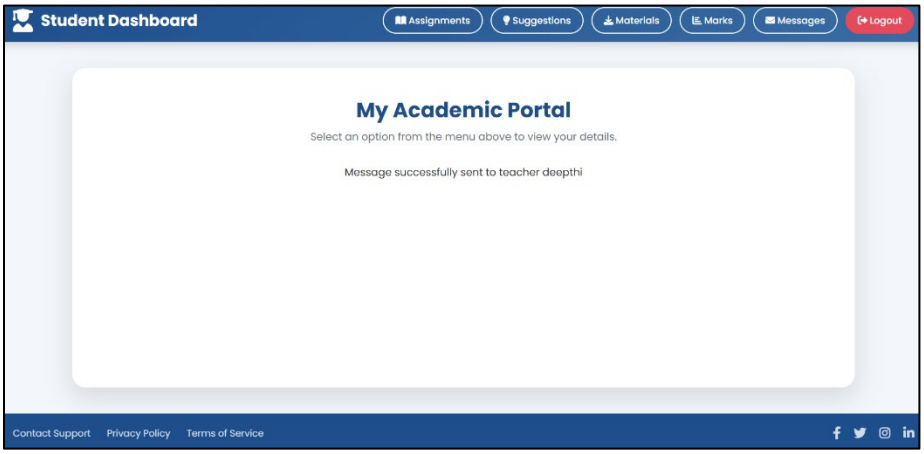
In above screen student can click on 'View Upcoming Assignments' link to view coming assignment list like below page



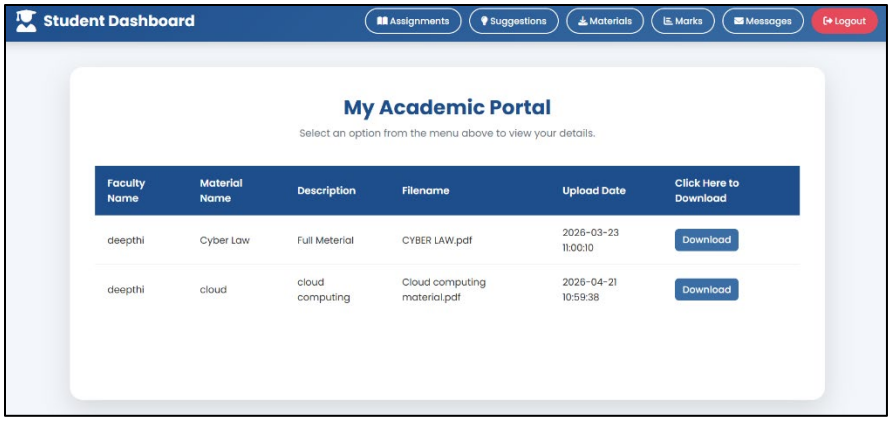
In above screen students can view upcoming assignments list and now click on ‘Messaging’ link to send message to faculty



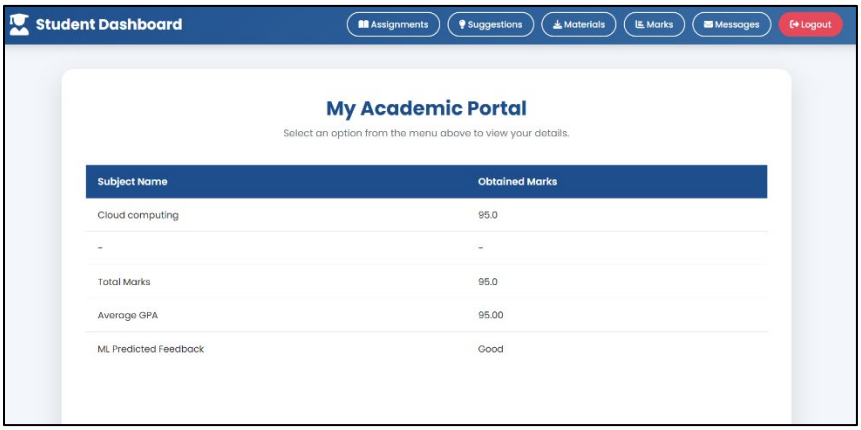
In above screen student will post doubts to teacher and then press button to get below page



In above screen message successfully sent to faculty and now click on ‘Download Material’ link to get below page



In above screen student can view list of available materials and can click on 'Download' link to download and in above screen in browser task bar can see material downloaded and now click on 'View Marks' link to view his marks and get below page



In above screen student will enter course and year and then press button to get below marks with ML feedback in above screen student can view all marks details and can see ML given feedback and now click on 'View Messaging' link to view messages from faculties.

FUTURE SCOPE

The Student Portal System developed in this project provides an efficient platform for academic management and student performance prediction. However, future enhancements can further improve system functionality and user experience. Advanced Artificial Intelligence (AI) and Machine Learning (ML) techniques can be integrated for more accurate student performance analysis and personalized learning support. Mobile applications for Android and iOS can also be developed to provide easy access to academic information, and communication features.

The system can be enhanced by adding online examination modules, digital classroom features, cloud integration, and advanced analytics dashboards. Future versions may also include facial recognition-based attendance systems, automated report generation, and chatbot support for student queries.

Regular security updates and system improvements will help maintain data protection and system reliability. These future enhancements will make the Student Portal System more intelligent, secure, scalable, and user-friendly for modern educational institutions.

CONCLUSION

The Student Portal System using Machine Learning provides an efficient and centralized platform for managing academic activities and monitoring student performance. The system simplifies tasks such as attendance management, assignment handling, marks entry, study material sharing, and communication between students, teachers, and administrators.

The integration of Machine Learning algorithms such as Naïve Bayes and XGBoost helps predict student performance accurately and provides useful academic feedback. The system reduces manual work, improves data management, and supports better academic decision-making.

The project was successfully implemented and tested, and all modules functioned correctly without major errors. Overall, the Student Portal System offers a reliable, secure, and user-friendly solution for modern educational management.

BIBLIOGRAPHY

1. Python Software Foundation, *Python Documentation*, Available at: https://www.python.org

2. Django Software Foundation, *Django Documentation*, Available at: https://www.djangoproject.com

3. Scikit-learn Developers, *Scikit-learn Machine Learning Library*, Available at: https://scikit-learn.org

4. Tianqi Chen and Carlos Guestrin, *XGBoost: A Scalable Tree Boosting System*, 2016.

5. Ian H. Witten, Eibe Frank, and Mark A. Hall, *Data Mining: Practical Machine Learning Tools and Techniques*, Morgan Kaufmann Publishers.

6. S. Kotsiantis, *Supervised Machine Learning: A Review of Classification Techniques*, Informatica Journal.

7. MySQL Documentation, *MySQL Database Management System*, Available at: https://www.mysql.com

References

1. Romero, C., & Ventura, S. (2010). *Educational data mining: A review of the state of the art*.IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews), 40(6), 601–618.

2. Baker, R. S. J. d., & Inventado, P. S. (2014). *Educational data mining and learning analytics*. In Learning analytics (pp. 61–75). Springer.

3. Kotsiantis, S. B., Pierrakeas, C. J., & Pintelas, P. E. (2004). *Predicting students' performance in distance learning using machine learning techniques*. Applied Artificial Intelligence, 18(5), 411–426.

4. Chen, T., & Guestrin, C. (2016). *XGBoost: A scalable tree boosting system*. In Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining (pp.785–794).

5. Yadav, S. K., Bharadwaj, B., & Pal, S. (2012). *Data mining applications: A comparative study for predicting student's performance*. International Journal of Innovative Technology and Creative Engineering, 1(12), 13–19. Relevance: Discusses comparison of machine learning models.